

Manual técnico de eAI Vehicular para DayZ Standalone

Framework de reproducción de intención humana para vehículos guiados por NPCs en DayZ Standalone — Versión v1.0

Importante — Proyecto público en desarrollo

Este documento describe la **versión v1.0** del framework, validada empíricamente con un servicio de transporte público de 24.92 km en Chernarus y generalizada a un segundo vehículo (Land Rover Defender). El framework es **funcional y operativo** en el estado actual.

Sin embargo, el proyecto continúa activo. Hay capacidades identificadas pero **no implementadas** (documentadas en la sección de "Trabajo en curso y roadmap" al final del documento), descubrimientos pendientes de incorporar al código, y casos de uso aún por explorar.

Próximas publicaciones: la semana siguiente a esta v1.0 esperamos publicar resultados adicionales (drill de curvas 90°, grabación de vuelta del bus, validación en terreno normal, prueba de invulnerabilidad del driver, etc.). El repositorio queda **abierto y en desarrollo activo** — invitamos a la comunidad a contribuir con observaciones, fixes, casos de uso nuevos.

Cómo leer este documento: el contenido está organizado por capítulos temáticos. Cada capítulo es autoconsistente; podés saltar al que te interese según tu caso de uso:

- **Si querés entender el breakthrough técnico:** Capítulos 1, 2, 3
 - **Si querés integrar un vehículo nuevo:** Capítulo 4 + Apéndice del vehicle profile
 - **Si vas a grabar rutas:** Capítulo 5.5 (buenas prácticas)
 - **Si querés evaluar si este framework es el correcto para tu caso de uso:**
alcance y límites (espectro fidelidad/naturalidad, dos umbrales del framework)
 - **Si sos un modder integrando con asistencia de IA:** leé el archivo
separado `MOD_CONTEXT_FOR_AI.md` y subilo a tu workspace junto con este manual
-

Sobre las anotaciones internas

Este documento incluye, además del contenido técnico publicable, **notas internas del proceso de desarrollo** marcadas con "Para el paper:" o similares. Estas marcas son artefactos de cómo se construyó el manual y pueden ser ignoradas por el lector. Eventualmente se limpian en revisiones futuras del documento.

Tabla de contenidos

1. Qué es eAI driving y qué posibilidades abre
 2. El breakthrough técnico: ShiftTo(FIRST) hardcoded
 3. Compatibilidad de vehículos: herencia automática
 4. Framework genérico
 5. Captura de rutas (PathLogger)
 6. Coexistencia con contenido del entorno
 7. Límites y gotchas de Enforce
 8. Buenas prácticas para grabar rutas
 9. Casos de uso vehiculares
 10. Casos de uso no vehiculares
 11. Apéndices (disclosure de IA, créditos, AI knowledge pack, autores)
 12. Trabajo en curso y roadmap (Sesión 2026-05-24)
-

Capítulo 1 — Qué es eAI driving y qué posibilidades abre

Definición

eAI driving es la capacidad de hacer que un personaje no-jugador (NPC) controlado por la IA del mod **Expansion-AI** opere un vehículo de DayZ Standalone con comportamiento útil: acelerar, frenar, cambiar marchas, seguir una ruta, parar donde corresponda, reaccionar a condiciones del entorno.

A diferencia del soporte vanilla de NPCs (que se limita a peatones que caminan o disparan), eAI driving habilita un nuevo eje de gameplay: **vehículos que se mueven solos con propósito**, sin necesidad de un jugador humano al volante.

Por qué importa: las posibilidades

Hasta hoy, los servers de DayZ Standalone que querían contenido vehicular dinámico tenían dos opciones: (a) dejar vehículos estáticos para que los jugadores los usen, (b) hacer eventos scripted que mueven vehículos como props (sin comportamiento, sin pasajeros). Con eAI driving funcional, se abren categorías de contenido que antes eran inaccesibles:

- **Transporte público** — buses, taxis, ferries con rutas regulares
que los jugadores pueden usar como medio de transporte real
- **Convoyes militares** — patrullas motorizadas con escolta, rutas
pre-grabadas, comportamiento defensivo ante amenazas
- **Caravanas comerciales** — traders ambulantes que recorren ciudades
con stock variable, llegando en horarios determinados
- **Servicios de logística** — ambulancia para emergencias médicas,

delivery de armamento a jugadores en misión, entregas a domicilio ordenadas por UI smartphone

- **Persecuciones / fugas** — vehículos AI que persiguen o huyen del jugador, con cambios de comportamiento según contexto
- **Misiones narrativas** — "acompañá al conductor AI hasta X", "interceptá al convoy antes de Y", "subí al vehículo del NPC misterioso que te lleva a una zona oculta"
- **Acceso exclusivo a zonas** — el vehículo AI como único medio para cruzar barreras de juego (zonas con CylinderTrigger de rebote para jugadores comunes, pero el vehículo AI tiene pase)
- **Carreras humano vs IA** — "ganale a Boris al punto X", showcase divertido del framework
- **Transporte marítimo** — ferries entre islas, excursiones, contrabando (en agua hay menos obstáculos, el pathfinding es más simple)
- **Vehículos de servicio para roles** — patrullas policiales, ambulancias, taxis que reaccionan a estado del jugador

Estas posibilidades dejan de ser ciencia ficción y pasan a ser implementables con el patrón documentado en este manual.

Qué encontramos al buscar información

Cuando empezamos a buscar cómo resolver eAI driving, consultamos los siguientes lugares:

- **Steam Workshop** — buscando mods de AI bus, transporte público, AI vehicles, AI patrol vehicular
- **GitHub público** — repositorios y forks de mods de transporte AI
- **Reddit** r/dayz y r/DayzMod
- **Foros oficiales de Bohemia / DayZ**

Lo que encontramos en esos canales:

1. **Mods abandonados** — repos con commits hasta cierto punto y después silencio. Screenshots prometedores en la página de publicación, pero el código no llegó a un estado funcional o los autores no lo documentaron.
2. **Mods funcionando parcialmente** — el bot se sube al vehículo, arranca el motor, pero queda clavado en primera marcha, no acelera más allá de ~40 km/h, o se sale de la ruta en la primera curva.
3. **Mods con limitaciones severas** — solo en líneas rectas, solo a velocidad reducida, o requieren el jugador presente para estabilizar.

Síntomas que vimos repetirse en esos intentos:

- El bot sale del vehículo apenas se sienta
- Motor a redline alto pero sin aceleración real (clavado en primera)
- Steering inestable con oscilaciones constantes
- Choques contra elementos del `.map` base de Chernarus

No encontramos documentación pública sobre cómo destrabar estos puntos.

La ausencia más significativa es en **Steam Workshop**, que es la vidriera natural del modding de DayZ: cualquier modder con una solución funcional tiene incentivos para publicarla ahí (visibilidad, suscriptores, reconocimiento de la comunidad). Si existiera un mod funcional de AI driving en condiciones, debería estar en Workshop. No lo está.

Es posible que la información circule en canales menos accesibles (Discord privados de modders, conversaciones uno-a-uno, código propietario no publicado). **No reclamamos ser los primeros en resolver esto** — solo afirmamos que para alguien que llega al problema y busca en los lugares donde naturalmente se busca, la solución no aparece disponible.

Nuestro objetivo con este manual es **llenar ese hueco de accesibilidad** — poner la información en un lugar donde el próximo modder con el mismo problema la encuentre en horas en vez de en semanas. Incluso si la solución existiera en algún canal privado, su no-disponibilidad pública es el problema que nosotros estamos resolviendo.

Si vos, lector, tenés conocimiento de otra solución previa publicada o accesible, **te agradecemos que nos lo indiques** — referenciar trabajo previo es parte de hacer las cosas bien. La meta es que la comunidad de modding tenga esta capacidad disponible, no que nosotros aparezcamos como descubridores.

Qué provee este documento

Este manual presenta:

1. **El breakthrough técnico** que destraba eAI driving (Capítulo 2)
2. **El alcance del patrón** — cómo se aplica automáticamente a la mayoría de los vehículos del juego sin código adicional (Capítulo 3)
3. **El framework genérico** — cómo abstraer el patrón para reusarlo en cualquier vehículo o caso de uso (Capítulo 4)
4. **Las decisiones de diseño de soporte** — captura de rutas, limpieza del entorno, calibración de manejo (Capítulos 4.4-4.5)
5. **Los gotchas no documentados** que vamos a encontrar al implementar esto en otros proyectos (Capítulo 5)
6. **Casos de uso concretos** — desde transporte hasta misiones narrativas, con propuestas de implementación (Capítulo 6-7)
7. **Disclosure metodológico** sobre el uso de IA en el desarrollo (Apéndice A)
8. **Una versión del documento optimizada para AI assistants** —

diseñada para que cualquier modder pueda meterla en su workspace de VSCode + Opus/GPT/Gemini y obtener asistencia con contexto completo del framework (Apéndice D)

Por qué este momento es relevante

DayZ Standalone está en versión 1.29 al momento de escribir esto, con el nuevo mapa oficial **Badlands** anunciado para próximas releases. Cada versión del juego y cada nuevo mapa expanden la superficie de contenido que puede beneficiarse de eAI driving. Documentar el patrón ahora, mientras todavía es novedoso, multiplica el impacto a futuro.

Capítulo 2 — El breakthrough: eAI hardcodea ShiftTo(FIRST)

Fecha de descubrimiento: 2026-05-18

Dónde:

@DayZ-Expansion-

Bundle/addons/ai_scripts/4_world/dayzexpansion_ai/entities/carscript.c líneas 88-111.

El hallazgo: la `modded class CarScript` de eAI sobrescribe `OnInput(dt)` y al final del método ejecuta literalmente:

```
CarGear gear = CarGear.FIRST;
if (turn < 0) {
    turn = -turn;
    gear = CarGear.REVERSE;
}
// ... mas calculos de throttle/steering ...
ShiftTo(gear);
```

Esto es una implementación minimalista de gear-handling: eAI prioriza poder mover el vehículo en absoluto (FIRST gear es suficiente para que el bot avance) sobre la sofisticación de cambios automáticos. Es una decisión razonable para su scope original (AI movement básico). El efecto colateral para mods que quieren shifts automáticos es que eAI fuerza `gear=FIRST` cada frame, sobrescribiendo cualquier `ShiftUp()` o `ShiftTo(N)` que un mod externo llame en otra parte del código.

Síntoma observable in-game: bus con motor a redline alto (>4500 RPM) que no acelera más allá de los ~40 km/h porque está clavado en primera. El diagnóstico definitivo fue nuestro, comparando el velocímetro manteniendo el vehículo en primera manualmente vs dejándolo al AI: ambos clamping en el mismo valor.

El fix: en lugar de pelear contra eAI desde afuera, **dejarlo ejecutar su lógica y sobrescribir el gear inmediatamente después** dentro del mismo `OnInput`. Misma estrategia que ya se usa típicamente para throttle/steering cuando un mod externo necesita controlar un vehículo conducido por AI.

```
// modded class CarScript del mod externo (no de eAI)
override void OnInput(float dt) {
```

```

super.OnInput(dt); // eAI ejecuta su OnInput, mete ShiftTo(FIRST)
if (!GetGame().IsServer()) return;
if (!ThisVehicleIsManagedByUs(this)) return;

// Aplicar inputs propios (sobreescribe lo que eAI puso en super)
SetThrottle(m_DesiredThrottle);
SetSteering(m_DesiredSteering);
SetBrake(m_DesiredBrake);
ShiftTo(m_DesiredGear); // <<< sobreescribe el ShiftTo(FIRST) de eAI
}

```

Arquitectura recomendada (single source of truth):

- Una clase service del lado del mod mantiene `m_DesiredGear` (int).
- Una lógica de AT (en `EOnPostSimulate` o equivalente) modifica `m_DesiredGear` basándose en RPM/redline con histéresis.
- `OnInput` simplemente lo enforza cada frame post-super.

Por qué importa para el lector: cualquier modder intentando hacer "AI vehicles" con eAI va a chocar con este bloqueo. Sin saber que el problema es la línea 111 del archivo de Expansion, queda en bucle infinito de hipótesis equivocadas (gear shift roto, motor sin torque, gearbox deshabilitado, etc).

Capítulo 3 — Compatibilidad de vehículos: herencia automática y adapters

Dónde vive realmente la "propiedad de ser manejado por bot"

Un instinto natural al ver el bus funcionando es preguntarse: "¿cómo le enseñamos a otros vehículos a ser manejados por la IA?" — esperando que la respuesta sea reescribir algo por cada vehículo. Pero la respuesta es más interesante: **esa propiedad no vive en ningún vehículo en particular**. Vive en **tres capas superpuestas** sobre la clase base `CarScript`:

Capa 1 — Engine vanilla de DayZ

La clase `CarScript` del engine define:

- El método `OnInput(float dt)` (hook ejecutado cada frame mientras hay un driver en el asiento)
- Los APIs base: `SetThrottle()`, `SetSteering()`, `SetBrake()`, `ShiftTo()`, `EngineStart()`, `GetGear()`, etc.

El `OnInput` vanilla espera que el input venga de un humano (teclado, joystick, controller). No sabe nada de IA.

Capa 2 — Expansion-AI (modded class `CarScript` en `ai_scripts.pbo`)

Vive en [@DayZ-Expansion-Bundle/addons/ai_scripts/4_world/dayzexpansion_ai/entities/carscript.c](#) . Sobrescribe `OnInput(dt)` para que, cuando el driver del vehículo es un `eAIBase` (en vez de un humano), lea del pathfinder de eAI y calcule throttle/steering/gear sintéticamente.

Esta es la capa que "enseña" a todos los cars del juego a ser manejables por IA, de una sola vez. Es el regalo invisible que da el mod de Expansion-AI a la comunidad.

También define el complemento del lado del bot: `eAIBase.StartCommand_Vehicle()` en `eaibase.c` línea 7407, que pone al eAI en un asiento de cualquier `Transport` .

Capa 3 — BrigadaZ_Transport (nuestro modded class CarScript)

En `scripts/4_World/BZBusCarScript.c` . Sobrescribe `OnInput(dt)` por tercera vez. La estructura:

```
override void OnInput(float dt) {
    super.OnInput(dt); // ejecuta la capa 2 (eAI)
    if (!GetGame().IsServer()) return;
    if (!ThisVehicleIsManagedByUs(this)) return;

    // ahora aplicamos nuestros valores ENCIMA de los de eAI
    SetThrottle(m_DesiredThrottle);
    SetSteering(m_DesiredSteering);
    SetBrake(m_DesiredBrake);
    ShiftTo(m_DesiredGear);
}
```

Herencia automática a toda la familia CarScript

Acá viene la parte poderosa: en Enforce, cuando se hace `modded class CarScript` , los cambios se aplican a **toda la jerarquía de clases que extienden de CarScript**, sin importar dónde se hayan definido (vanilla o mod externo).

Concretamente:

CarScript (engine)	
├ Hatchback_02 (vanilla)	← hereda capas 1+2+3
├ Sedan_02 (vanilla)	← hereda capas 1+2+3
├ Ada_4x4 (vanilla)	← hereda capas 1+2+3
├ Olga_24 (vanilla)	← hereda capas 1+2+3
├ Truck_01 (vanilla)	← hereda capas 1+2+3
├ M3S_Covered (vanilla)	← hereda capas 1+2+3
├ ExpansionBus (mod Expansion)	← hereda capas 1+2+3 (lo que usamos)
├ Sport_Car (mod Workshop X)	← hereda capas 1+2+3
├ LandRover (mod Expansion)	← hereda capas 1+2+3
├ PickupTruck (mod Workshop Y)	← hereda capas 1+2+3
├ ...	← cualquier futuro mod que extienda CarScript

Todos los vehículos de la lista ya saben ser manejados por bot, gracias a las tres capas combinadas. No hay que enseñarle a cada uno individualmente. Lo único que cambia entre uno y otro es la configuración (gearbox, redline, peso, fuel tank, posición de asientos) y el modelo 3D. El **comportamiento** de "yo, vehículo, puedo recibir input de eAI y aplicarlo" está en la clase padre y se hereda gratis.

Más todavía: **si mañana sale un mod nuevo que agrega "Mod-X-Bus" como `extends CarScript`, nuestro framework lo controla automáticamente** sin que ni el autor del mod ni nosotros hagamos nada extra. Esa es la potencia del patrón.

Cuándo se rompe la herencia: 5 casos

No todos los vehículos del juego entran en la jerarquía de `CarScript`. De más común a más raro:

Caso 1 — El mod no es un coche aunque lo parezca

Vehículos que pertenecen a otras familias de clase base:

- **Botes** (@MaharlikaPH_Boats, @gebsfish) → `BoatScript`, no `CarScript`
- **Helicópteros** (@DayZ-Expansion-Licensed Bell412, etc.) → `Helicopter`
- **Bicicletas vanilla** → `BicycleEntity`
- **Tanques con orugas** → algunos mods los hacen con base custom porque las orugas no son ruedas
- **Hovercraft / aerodeslizadores** → custom physics, custom base

Para estos no es que "el autor se equivocó". Genuinamente no son cars. Cada familia necesita su propio adapter (ver más abajo).

Caso 2 — Mod viejo de antes del refactor de DayZ

DayZ pasó por varios refactoros grandes de su clase de vehículos:

- Pre-2018: clases distintas, `Transport` era más genérico
- 2019-2021: introducción de `CarScript` como base estándar
- 2022+: nuevas APIs (`ShiftTo`, getters de input)

Mods abandonados de hace 4-5 años pueden estar usando una jerarquía vieja que sigue compilando pero no implementa todo lo que esperamos. Síntoma típico: el bot se sube, el motor enciende, pero `SetThrottle()` no produce efecto.

Caso 3 — El autor metió override de `OnInput` sin llamar a super

Esto sí es un bug del autor, no su elección de clase base. El vehículo extiende de `CarScript` (bien), pero el autor escribió:

```
class MyCoolCar extends CarScript {  
    override void OnInput(float dt) {  
        // mi propia lógica acá  
    }  
}
```

```
// ... pero NO llamo a super.OnInput(dt)
}
}
```

Cuando esto pasa, la cadena `super` queda cortada. Las capas 2 (eAI) y 3 (nuestra) no se ejecutan para este vehículo en particular. Boris se sube pero no responde.

Fix: el autor agrega `super.OnInput(dt)` como primera línea de su override. Una línea, fix instantáneo. Como reportar bug.

Caso 4 — Vehículo desde una clase totalmente distinta

Casos exóticos del Workshop:

- Jetpacks o gliders modeados como `extends EntityAI` directo
- "Vehículos" que en realidad son zombies/animales modificados
- Vehículos-teletransportadores que son props con scripts custom

Estos no participan de la jerarquía de `Transport` para nada. Nuestro framework no los toca y tampoco debería — no son vehículos en sentido convencional.

Caso 5 — El autor hizo su propia jerarquía paralela

Algunos mods grandes hacen su propia "platform" de vehículos:

- Sus vehículos extienden de un `MyMod_BaseVehicle` que NO extiende de `CarScript`
- Toda la lógica de gearbox / wheels / steering la implementaron ellos desde cero
- Razones típicas: simulación más realista que la del engine, o cubrir bugs de la implementación oficial

Si esto pasara con un mod popular, el autor tendría su propia API de "set throttle / set steering" pero no es la estándar. Habría que escribir un adapter específico para ese mod si valiera la pena.

Cómo verificar rápido si un vehículo entra en la jerarquía

Tres heurísticas, de menos a más confiable:

Heurística 1 — Mirar el config.cpp del mod

En `class CfgVehicles` del mod:

```
class MyMod_Car: Car { ... }      ← Car base, casi seguro CarScript
class MyMod_Car: CarScript { ... } ← explícito, sí
class MyMod_Car: Transport { ... } ← Transport base, NO es CarScript
class MyMod_Car: Inventory_Base { ... } ← muy mal, no es vehículo
```

Limitación: esta es la jerarquía de **config**, no la de **script**. No siempre coinciden 1 a 1.

Heurística 2 — Buscar el .c del mod

Si el mod publica scripts:

```
class MyMod_Car extends CarScript { ... }    ← compatible
class MyMod_Car extends Car { ... }          ← Car es padre de CarScript, también compatible
class MyMod_Car extends EntityAI { ... }     ← NO está en la jerarquía
```

Heurística 3 — Test empírico in-game (más confiable)

Spawnear el vehículo + spawnear un eAI + intentar bordarlo con `StartCommand_Vehicle()`. Si el bot se sube y acepta input → es CarScript-compatible. Si no → no es.

En este proyecto, la hotkey NUMPAD 1 (debug) hace exactamente esto. Cambiando el classname del vehículo objetivo, se puede probar cualquier vehículo modded en 5 segundos.

Estimación práctica del % de mods compatibles

Para los mods activos del server BrigadaZ al momento de escribir esto:

Mod	Probable compatibilidad
Car packs grandes del Workshop (mods con N vehículos)	✓ CarScript (patrón estándar)
Mods de retexturado	✓ no agregan vehículos, no relevantes
Mods de barcos (BoatScript)	✗ BoatScript — needs boat adapter
@DayZ-Expansion vehículos (autos, vans)	✓ CarScript / Car
@DayZ-Expansion helicópteros (Gyro, Hatchbird, Bell412)	✗ Helicopter — needs heli adapter
Mods de animales / fauna	n/a no es vehículo
Pickup trucks o vehículos custom del Workshop	✓ CarScript probable

Estimación rápida del Workshop general: **el 95% de los car mods están en la jerarquía CarScript** — si no lo estuvieran, ni siquiera podrían subirse al volante con un eAI bot, porque eAI's modded class CarScript no aplicaría. Los autores de mods que quieren que sus vehículos sean manejables ya están alineados con la jerarquía estándar porque sino sus vehículos no funcionarían ni siquiera con humanos en algunos casos.

Adapters: cuando hay que escribir uno

Para las familias que NO heredan de `CarScript`, el patrón del framework se replica con una clase adapter específica:

Adapter de BoatScript (pendiente — v1.2)

`modded class BoatScript`, misma estructura que nuestro `modded CarScript` pero más simple porque los botes no tienen gearbox:

```
modded class BoatScript {
    override void OnInput(float dt) {
        super.OnInput(dt);
        if (!GetGame().IsServer()) return;
        if (!BoatService.GetInstance().IsManaged(this)) return;

        // botes: throttle y rudder (que es como steering)
        SetThrustControl(m_DesiredThrust);
        SetSteeringControl(m_DesiredRudder);
        // sin gear, sin brake – el bote se frena soltando throttle
    }
}
```

(Los nombres exactos de los métodos (`SetThrustControl`, etc.) hay que verificarlos cuando se escriba el adapter — son guess basados en la API esperable.)

Adapter de Helicopter (pendiente — v2.0+)

Mucho más complejo. Cuatro ejes de control:

- Collective (subir / bajar)
- Cyclic forward (inclinación adelante / atrás)
- Cyclic lateral (inclinación izquierda / derecha)
- Anti-torque / yaw (rotación del rotor de cola)

Además, navegación 3D — el waypoint ya no es un punto en el plano (x, z) sino un punto en el espacio (x, y, z) con altura controlada activamente. Esto requiere un sistema de pathfinding distinto al de los cars, que salen del scope de este manual.

Cierre del capítulo

La lectura optimista: **el framework ya cubre el 95% de los vehículos del juego sin escribir una línea más**. La lectura realista: lo otro 5% (botes, helis, exóticos) requiere adapters específicos pero el patrón está claro y se replica con esfuerzo bounded.

Capítulo 4 — Framework genérico

(pendiente — refactor de `BZBusService` → `BZAIVehicleService`.)

Capítulo 4.4 — Captura de rutas: del waypoint manual al recording paramétrico

Antes de llegar al sistema actual, el problema de "dónde exactamente debe ir el vehículo" en una ruta larga (15 paradas, ~30 km de costa) era casi prohibitivo. La evolución del enfoque:

Iteración 1 — Marcar waypoints a mano (descartada)

Plan original: abrir el mapa, identificar las 15 paradas + puntos intermedios cada N metros, anotar coordenadas a mano, pegarlas en un JSON.

Por qué no funcionaba:

- Una ruta de 30 km con waypoints cada 20m requiere ~1,500 puntos.
- Las alturas (componente Y) no son obvias en mapas 2D.
- Las curvas suaves requieren waypoints densos para que el AI driver no oscile.
- Cada cambio de ruta o cada vez que se ajusta una parada implica re-marcar todo.

Iteración 2 — Generar movimientos sintéticos por algoritmo (descartada)

Idea: tomar la línea de la costa, samplear cada N metros, calcular heading entre puntos consecutivos, escupir el JSON automáticamente.

El resultado era geométricamente preciso pero ignorante de la realidad del manejo: no tenía noción de cuándo frenar antes de una curva, cuándo acelerar al salir, dónde cambiar de marcha, dónde es seguro tomar la línea interna y dónde no. El playback se sentía robótico.

Iteración 3 — PathLogger: imitar grabación humana

La solución que terminó funcionando: un service client-side que durante una sesión de manejo manual graba, a 4 muestras por segundo (cada 250ms), parámetros del vehículo que después se replican en el playback.

Parámetros capturados:

Campo CSV	Origen	Uso en playback
time_s	tiempo desde inicio rec	diagnóstico
x, y, z	<code>Car.GetPosition()</code>	waypoint
heading_deg	derivado de <code>Car.GetDirection()</code>	referencia
speed_kmh	<code>Car.GetSpeedometerAbsolute()</code>	<code>targetSpeed</code>
is_stop	flag manual via hotkey	marca de parada

Campo CSV	Origen	Uso en playback
gear	<code>Car.GetGear()</code>	<code>targetGear</code>
throttle	<code>Car.GetThrottle()</code>	<code>targetThrottle</code>
brake	<code>Car.GetBrake()</code>	<code>targetBrake</code>

Hotkeys de operación

Tecla	Acción
NUMPAD 2	Toggle servicio del bus (spawn / respawn de la AI driver)
NUMPAD 5	Toggle start/stop de la grabación
NUMPAD 6	Pause/resume sin cerrar el archivo
NUMPAD 4	Marcar parada en el sample actual (<code>is_stop=1</code>)

Los datos quedan en CSV en `$profile:BrigadaZ_Transport_PathLogger/` (en el cliente, no en el server, porque PathLogger corre client-side). Un script externo de PowerShell convierte el CSV en el JSON que consume el bus: `profiles/BrigadaZ_Transport/BZBusRoute.json`.

Posibles extensiones del PathLogger

Lo que ya captura cubre la mayoría de la fidelidad humana. Lo que falta y se puede sumar para casos más exigentes:

- **Handbrake** — útil para vehículos donde se usa en derrapes controlados (no aplica al bus pero sí a sedans en algunos casos).
- **Distancia al borde de la ruta** — para reproducir lateralidad (¿se manejaba pegado a la línea blanca, al medio del carril, al borde derecho?).
- **Eventos custom** — bocinazo, encender / apagar luces, abrir puerta, acciones que el playback puede reproducir en el waypoint exacto.
- **Estado del jugador secundario** — si se grabó manejando con copiloto, capturar el estado del segundo asiento.
- **Modo "automático" del vehículo durante grabación** (planeado v1.1):
hacer que el vehículo manejado por humano durante una sesión de PathLogger tenga shift automático (incluso si en uso normal el jugador maneja a mano). Esto elimina al "operador olvidándose de bajar marchas al frenar" como fuente de grabaciones sub-óptimas. La AT que ya tenemos en el modded CarScript podría activarse condicionalmente cuando detecta una grabación PathLogger en curso. Alternativa: usar un vehículo modded con AT nativa solo durante recording. Resultado esperado: grabaciones más limpias sin pedirle al operador habilidad de chofer manual.
- **PathLogger adaptado para patrullas a pie y acciones no-vehiculares:**

el mismo principio de "grabar el movimiento humano, reproducir con AI" sirve más allá de vehículos. Patrullas de eAI a pie actualmente son puntos discretos conectados linealmente o movimiento aleatorio en un radio. Una versión del PathLogger adaptada para capturar movimiento de jugador caminando (posición, velocidad, dirección, postura) y reproducirla en un eAI a pie daría **patrullas con movimiento más natural**: rutas no rectilíneas, pausas en puntos de interés, cambios de pace, comportamientos contextuales que hoy no se pueden replicar con waypoints simples.

Por qué importa para el lector

La captura de rutas es un sub-problema independiente del framework principal. Si tu mod no es "vehículo con ruta fija" — por ejemplo, si es "patrullaje aleatorio en zona X" o "convergencia a un punto desde spawn variable" — no necesitas PathLogger. Pero si tu caso es reproducir una secuencia precisa de comportamiento humano en un vehículo controlado por AI, el patrón descrito acá es probablemente el más simple que da resultados aceptables sin requerir IA generativa ni machine learning.

Idea futura: PathLogger con perfil del conductor

Una observación que surgió durante el desarrollo y que dejamos como roadmap para v2 o más adelante: **cada conductor tiene una "huella" propia de manejo** — qué tan agresivamente frena, qué tan abiertas o cerradas toma las curvas, dónde empieza a decelerar antes de una parada, qué tan pronto cambia de marcha al acelerar. Hoy el PathLogger captura una sola grabación y el playback la reproduce literal, asumiendo que esa grabación es "la" verdad.

Una versión más sofisticada podría:

1. **Capturar N grabaciones del mismo conductor** sobre la misma ruta
o tramos parecidos
2. **Computar un perfil estadístico** del conductor: deceleración
promedio antes de parar, distancia típica de anticipación a curvas, variabilidad de pisada, etc.
3. **Aplicar el perfil al playback** como ajuste fino sobre cualquier
grabación, suavizando outliers (un volantazo accidental que el conductor no haría normalmente) o reforzando patrones consistentes
4. **Permitir múltiples perfiles de conductor** que el server admin
selecciona según el caso de uso ("conductor cuidadoso", "conductor rápido", "conductor de bus profesional")

Esto se acerca al territorio de "el framework conoce a su operador", que es un paso natural si el proyecto crece. No es necesario para v1.0 pero es la clase de mejora que podría diferenciar futuras versiones.

La intuición original (atribuida a Sonom4n durante una sesión de calibración del bus): *"el PathLogger primero tiene que conocerte y después aplicar la corrección para cada conductor"*.

Capítulo 4.5 — Coexistencia con contenido del entorno

Lo primero a entender: el framework convive con cualquier entorno

Una pregunta natural al ver el bus circulando es: *"¿hace falta limpiar todo el mapa de objetos para que el AI no se choque?"*. La respuesta es **no**. El PathLogger graba las rutas que vos manejás manualmente, y vos como humano ya estás esquivando los obstáculos del mapa cuando grabás. Cuando el bus reproduce esa ruta, sigue tus trayectorias exactas — incluyendo los pequeños esquivos que vos hiciste alrededor de un wreck o de un árbol caído.

Es decir: si tu server tiene Expansion-Bundle con su módulo decorativo activo (`ExpansionWorldObjectsModule`), el AI vehicle **puede circular perfectamente** entre ese contenido, siempre y cuando la grabación que hiciste lo tenga en cuenta. El framework no necesita un mundo "limpiado" para funcionar.

Nota de contexto: nuestra relación con Expansion

Expansion-Bundle es el cimiento sobre el cual todo este framework existe. Sin el trabajo del equipo de Expansion — particularmente el sub-módulo eAI, que hace posible toda la categoría de "AI vehicles" en DayZ — este proyecto sencillamente no habría sido escribible. Y el módulo decorativo del Bundle es un aporte valioso para la mayoría de los servers Expansion: aporta el "look and feel" post-apocalíptico característico que la comunidad asocia con esa experiencia.

Por eso este capítulo no es "cómo sacar la decoración de Expansion" — es "cómo coordinar con la decoración de Expansion para tu caso de uso específico". La mayoría de los servers no necesitan hacer nada.

Cuándo Sí aplica una limpieza explícita

Hay un caso particular donde nos resultó conveniente hacer una limpieza más amplia: cuando la **ruta del AI vehicle es muy trafficada por contenido decorativo denso** (en nuestro caso, la coastal road de Chernarus tiene literalmente cientos de wrecks distribuidos a lo largo de los 30km que recorre el bus), y querés priorizar **fluidez visual y de pathfinding** sobre la fidelidad de la decoración.

En nuestro caso particular: la coastal road es muy densa en contenido decorativo, queremos que el bus sea un "showcase fluido" del framework para los usuarios del server, y la limpieza de esa zona nos da el resultado visual y técnico que buscábamos.

Para muchos otros casos de uso este nivel de limpieza es innecesario:

- **Patrullas militares en zonas alejadas:** el contenido decorativo ayuda a la atmósfera, el AI puede esquivar
- **Caravanas que recorren poco terreno:** la grabación absorbe los esquivos, no hace falta limpiar
- **Servicios marítimos:** el agua no tiene decoración densa
- **AI vehicles que circulan ocasionalmente:** el modder graba con

cuidado y listo

Si no aplica tu caso de uso, podés saltar la mayor parte de este capítulo. Solo la Capa 5 (el `.dze` para wrecks del `.map` base de Chernarus) puede ser relevante si los wrecks específicos están EXACTAMENTE encima de un waypoint y bloquean físicamente al vehículo. La Capa 5 es un caso quirúrgico, no masivo.

Lo que documentamos a continuación

Las 5 capas siguientes describen **lo que nosotros hicimos** para nuestro caso particular. Es una guía para alguien que tenga un caso similar (AI vehicles en ruta densa de Chernarus con Expansion-Bundle). Es referencia documental, no checklist obligatorio.

La configuración se aplica en **5 capas** ordenadas de más a menos amplias en alcance. Quien quiera saltar este capítulo y volver después no se pierde el core del framework.

Capa 1 — Configurar `ExpansionWorldObjectsModule` (boolean único)

Dónde: `profiles/ExpansionMod/Settings/GeneralSettings.json`

```
"Mapping": {  
  "UseCustomMappingModule": 0,  
  "Mapping": [ ... ]  
}
```

Qué hace este setting: desactiva la carga del módulo decorativo de Expansion que coloca ~41,000 elementos atmosféricos (wrecks, basura, plantas, vegetación, decals) en coordenadas fijas durante `OnMissionStart`. La lógica vive en `ExpansionWorldObjectsModule.LoadObjects` en `scripts.pbo` del Bundle (clase static, configurable por este boolean pero no extensible vía `modded class`).

Por qué elegimos desactivarlo en nuestro caso: el módulo decorativo agrega contenido visual a lo largo de la coastal road que es muy valorable como atmósfera para muchos servers. Para nuestro showcase específico del AI bus, priorizamos **fluidez visual** (que el bus pase por la costa sin que el escenario tenga clusters densos de wrecks alrededor) sobre la decoración. Es una decisión de *compromiso* entre dos features valiosas (decoración densa vs ruta visualmente despejada para el AI vehicle), no un juicio sobre el módulo en sí.

Un server que use este framework para un caso de uso menos sensible a la densidad visual (patrulla militar en zonas alejadas, caravana ocasional, etc.) puede dejar el módulo activado sin problema.

Bonus técnico: el boolean es global — afecta a todo el mapa, no solo la coastal road. Eso es deliberado de Expansion (la API no permite desactivar por zona). En nuestro caso eso nos sirve porque queremos AI driving en toda la costa, pero un server que quiera mantener el contenido decorativo en zonas alejadas del path tendría que usar la configuración granular `Mapping[]` del mismo archivo en lugar del boolean global.

Beneficios secundarios (no centrales a nuestro objetivo pero valen mencionar): performance del server mejora levemente (menos objetos = menos colliders = menos carga), startup es un poco más rápido. Estos son efectos colaterales — la razón principal para nosotros es la compatibilidad con AI vehicles.

Capa 2 — Purga de mapgrouppos.xml

Dónde: `mpmissions/dayzOffline.chernarusplus/mapgrouppos.xml`

Qué hacer: borrar todos los bloques `<group name="Land_Wreck_*">` y variantes case-insensitive:

- `Land_Wreck_*`
- `Land_wreck_truck01_*`
- `bldr_wreck_*`
- `Expansion_Wreck_*`

En este proyecto: ~1,335 grupos eliminados. Tamaño del archivo: 1495 KB → 1312 KB.

Por qué importa: `mapgrouppos.xml` define dónde el CE puede spawnear los grupos definidos en `mapgroupproto.xml`. Aunque el classname tenga nominal=0 en `types.xml`, ciertos sistemas legacy pueden seguir respawneando desde este archivo si no se purga.

Pitfall: comparaciones case-sensitive. Asegurar que el script de purga sea case-insensitive o ejecutar múltiples pasadas con cada variante de capitalización.

Backup: crear `.bak` antes de tocar.

Capa 3 — Desactivar eventos Static en events.xml

Dónde: `mpmissions/dayzOffline.chernarusplus/db/events.xml`

Qué hacer: poner `active="0"` en:

- `StaticMilitaryConvoy` (254 wrecks dinámicos)
- `StaticPoliceSituation` (253 wrecks dinámicos, también roadblocks de madera — sacrificio aceptable)

Dejar activos:

- `StaticHeliCrash` (loot militar útil, no toca rutas terrestres)
- `StaticTrain` (manejado en capa 4 — purga selectiva, no desactivación total)

Por qué importa: estos eventos generan wrecks dinámicos por el CE en cada ciclo. Sin esta capa, los wrecks reaparecen después de limpiar el mapa.

Capa 4 — Purga selectiva en cfgeventspawns.xml

Dónde: `mpmissions/dayzOffline.chernarusplus/cfgeventspawns.xml`

Qué hacer: del bloque `<event name="StaticTrain">`, borrar las `<pos>` que caigan dentro de la banda de la ruta del vehículo AI. En este proyecto: 11 posiciones eliminadas dentro de banda de 200m de la coastal road. Quedan 10 posiciones de `StaticTrain` en el interior del mapa.

Por qué importa: `StaticTrain` no es 100% prescindible (mejora atmósfera en zonas alejadas), pero los trenes wreckeados dentro de la ruta del bus son obstáculos críticos. Purga selectiva es preferible a desactivación total.

Relación con cfgeventgroups.xml: este archivo define QUÉ spawna cada group; cfgeventspawns.xml define DÓNDE. Para desactivar wrecks de eventos hay tres tornillos disponibles: events.xml (active), cfgeventspawns.xml (pos), cfgeventgroups.xml (children).

Capa 5 — DayZ Editor Loader (.dze) para wrecks del .map base

Dónde: mpmissions/dayzOffline.chernarusplus/EditorFiles/*.dze (cargado por @DayZ Editor Loader)

Por qué hace falta: Chernarus tiene wrecks hardcoded en el .map binario del mundo. Estos NO se pueden borrar con `GetGame().ObjectDelete(obj)` — el método retorna sin error pero el engine los restituye al siguiente chunk load. Hay que usar `SuppressedObjectManager.Suppress(obj)` de DayZ Editor Loader. El .dze ejecuta esto al iniciar el server para los objetos que el admin marcó visualmente en el editor.

Cómo se hace:

1. Iniciar DayZ con @DayZ Editor Loader activo
2. Entrar al editor (in-game tool)
3. Seleccionar visualmente cada wreck del .map base que estorbe la ruta
4. Marcar como delete
5. Guardar el .dze
6. Copiar a EditorFiles/ del server

En este proyecto: BusS_A.dze con 27 wrecks marcados manualmente.

Cuándo es necesario: después de las capas 1-4, lo que quede son los wrecks del .map base. Esos son los que se atacan con .dze. Casi siempre son pocos (< 50 a lo largo de una ruta de 30km).

Red de seguridad: BZRouteCleanup.c (sweep al startup)

Dónde: [scripts/4 World/BZRouteCleanup.c](#)

- [data/wrecks_cleanup.json](#)

Qué hace:

- Corre al startup del server con +5s de delay
- Lee wrecks_cleanup.json con anchors + patterns + radius/stride
- Interpola una banda a lo largo de la ruta (39 anchors, cada 20m, radio 25m)
- En cada punto: `GetGame().GetObjectsAtPosition3D(center, radius, objects, proxies)`
- Para cada objeto que matchee patterns (Land_Wreck_ , bldr_wreck_ , Expansion_Wreck_): `GetDayZGame().GetSuppressedObjectManager().Suppress(obj)`
- One-shot: corre una sola vez por arranque, ~700ms

Por qué es necesaria aunque tengamos las capas 1-5: en el último arranque encontró 512 wrecks. La red de seguridad atrapa lo que se cuela por spawns nuevos, mods agregados, parches de Expansion, etc.

Pitfall: los patterns son case-sensitive en script Enforce (`IndexOf` no es case-insensitive sin flag). TODO de este proyecto: cubrir variantes `Land_wreck_` vs `Land_Wreck_`.

Resumen: 6 mecanismos, 5 archivos de config + 1 script

Capa	Mecanismo	Cuándo aplica
1	UseCustomMappingModule=0	Si usás Expansion-Bundle
2	mapgrouppos.xml purga	Siempre
3	events.xml deactivate	Siempre
4	cfgeventspawns.xml purga selectiva	Si tu ruta toca StaticTrain
5	.dze con Suppress	Para wrecks del .map base
★	Sweep script en startup	Red de seguridad

Para el lector que llegó hasta acá

Si llegaste a este capítulo es porque ya entendiste el framework principal y querés saber por qué tu AI driver se sigue chocando con cosas que aparentan ser invisibles. La respuesta casi siempre está en una de estas 5 capas: hay algún elemento del mundo (decorativo, de evento, o del map base) que está en el path y que el framework no puede esquivar.

El cleanup .dze que distribuimos con el mod es ilustrativo, no prescriptivo. Sirve como ejemplo de "cómo se limpia terreno para una ruta de AI vehicle", pero está hecho para NUESTRO server (remueve también algunas paradas vanilla porque las reemplazamos con las del mod externo @Brutalist Bus Stops). Cualquier admin que use el framework debería **hacer su propio cleanup .dze** según las necesidades específicas de su server y su ruta. El nuestro es solo una muestra de la técnica.

No es nuestro aporte original — la configuración del entorno de un server es algo que cualquier admin hace para adaptar su mundo a sus casos de uso particulares. Lo documentamos acá porque fue parte de nuestro camino y porque, para alguien que quiera replicar este framework en su propio setup, ahorra varias horas de debugging preguntándose por qué el bus "se chocó con nada".

Y un recordatorio final que mencionamos al principio: **eAI (sub-módulo de Expansion) es lo que hace posible este framework**. Toda esta sección documenta coordinación con OTROS módulos de Expansion para nuestro caso de uso específico, no crítica al diseño del paquete.

Capítulo 5 — Limites y gotchas de Enforce relevantes para esta clase de proyectos

Gotcha 5.1 — "Formula too complex" en concatenaciones largas

Fecha de descubrimiento: 2026-05-18 (y antes 2026-05-17, ya pasó dos veces durante el desarrollo del framework).

Dónde: BZPathLogService.c línea 160 (versión que agregó `throttle` y `brake` al CSV).

Síntoma: al compilar el script, RPT muestra:

```
SCRIPT (E): file.c,N: Formula too complex
SCRIPT (E): file.c,N: Incompatible parameter 'X'
```

(El segundo error es un síntoma secundario — el parser se rinde y reporta tipos confundidos.)

Reproducción mínima:

```
// Esto explota:
string line = "" + a + "," + b + "," + c + "," + d + "," + e
              + "," + f + "," + g + "," + h + "," + i + "," + j;
```

Causa raíz: Enforce tiene un límite no documentado en la cantidad de operandos encadenados con `+` en una sola expresión. Empíricamente, alrededor de 9 operandos pasa el límite y el compilador colapsa. Es probable que se deba a profundidad de stack del parser, pero esto no está confirmado en la documentación oficial.

Fix:

```
string line = "" + a + "," + b + "," + c;
line += "," + d + "," + e + "," + f;
line += "," + g + "," + h + "," + i + "," + j;
```

Regla práctica: si una expresión de string concat pasa de 6-7 operandos, ya conviene dividir aunque todavía no haya explotado. La próxima edición casi siempre lo lleva al límite.

Por qué importa para el lector: afecta a cualquier proyecto que escriba telemetría a CSV, logs estructurados, o líneas JSON construidas manualmente. Es una pérdida de tiempo enorme la primera vez que aparece porque el mensaje de error no apunta a la causa real.

Gotcha 5.2 — Clases del engine no admiten `modded class`

(documentado en notas previas, agregar aquí cuando pulamos el manual.)

Gotcha 5.x — Centroide de waypoints corta curvas por adentro

Fecha de descubrimiento: 2026-05-19

Síntoma: durante el desarrollo del steering del bus, una variante "natural" del pure pursuit que parece tener sentido es apuntar el AI hacia el **centroide (promedio) de los próximos N waypoints**, con la idea de que esto da una anticipación adaptativa (~1s a 4 waypoints × 250ms/sample).

En la práctica **falla en curvas**: el centroide de puntos sobre una curva queda **dentro** de la curva (más cerca de la cuerda que del trazado). El AI siguiendo el centroide corta la curva por adentro, lo que en un mundo con obstáculos = choque garantizado.

Reproducción: implementar steering apuntando al centroide de N waypoints adelante. Probar en una curva cerrada. El vehículo va a recortar la curva.

Fix: usar pure pursuit clásico — apuntar a un punto **interpolado sobre la ruta** a distancia fija (e.g., 10m). El punto está garantizado sobre el path, no inside. La trayectoria respeta el trazado en curvas a cambio de un poco menos de "anticipación".

```
// MAL – centroide cuts corners:
vector lookahead = AverageOf(next4Waypoints);

// BIEN – interpola sobre el path a distancia fija:
vector lookahead = InterpolateAlongPath(LOOKAHEAD_DIST);
```

Por qué importa para el lector: la intuición de "el promedio de waypoints da una posición sintética buena" es muy fuerte y atractiva. Vale la pena explicitar que la geometría no acompaña a esa intuición. El pure pursuit clásico (en uso en robótica desde los 80s) ya resolvió este problema apuntando sobre el path.

Gotcha 5.3 — `new ClassName(arg)` no funciona en Enforce

(idem — crear vacío y setear campos.)

Gotcha 5.4 — Orden de scopes 3_Game → 4_World → 5_Mission

(idem — helpers compartidos al scope más temprano.)

Capítulo 5.5 — Buenas prácticas para grabar rutas (PathLogger)

Fecha: 2026-05-19. Algo que articulamos durante el desarrollo: el playback del AI no busca clonar a un humano específico, sino reproducir un manejo "AI-friendly" donde humano y máquina convergen.

El éxito del playback depende tanto del framework como de **cómo grabaste**. Una grabación con volantazos bruscos, pulsaciones de freno y cambios constantes va a dar un playback nervioso aunque el código sea perfecto.

Steering

- **Líneas anchas en curvas**. El AI persigue waypoints y reacciona. Líneas cerradas internas crean ángulos agudos entre waypoints consecutivos que el AI traduce a oscilación.

- **Correcciones suaves y largas**, no volantazos puntuales. Si el bus se desvía levemente, ajustar gradualmente.
- **Manos quietas en rectas**. Cualquier microvolantazo "para ocupar tiempo" queda grabado.

Freno

- **Frenadas sostenidas, no pulsadas**. Un $\text{brake}=0.5$ durante 3s es trivial de reproducir; tres pulsos de $\text{brake}=0.8$ generan discontinuidades.
- **Empezar gradual**. Curva exponencial (creciente) > lineal > instantánea.
- **Evitar handbrake**. No se captura en la API estándar de `Car`.

Throttle

- **Sostener velocidad** sin pisar-soltar. Si querés ir a 50 km/h, mantené la pisada que produce 50, no hagas micro-correcciones constantes.
- **Sin cruise control** si el vehículo lo tiene. Pisada manual sostenida.

Cambios

- **Minimizar shifts**. Si una marcha aguanta sin redlinear, dejala. La AT del AI puede cambiar en puntos cercanos pero no idénticos a los tuyos, y cada divergencia es un microbache.
- **Cambiar en rectas**, no en curvas. Si hay que cambiar marcha cerca de una curva, hacerlo antes o después, no en el medio.

Paradas

- **Frenar siempre en el mismo punto** (idealmente bus alineado con cartel). La inconsistencia entre paradas queda grabada como inconsistencia.
- **Quietud completa de al menos 1s** antes de tocar MarkStop, así el sample escrito tiene velocidad=0 limpia.

Generales

- **Manejar en 3ra persona** para mejor referencia visual de la trayectoria del vehículo, que es lo que el AI va a "ver".
- **Mentalidad de chofer profesional**, no de rally. Suavidad por encima de velocidad.

- **Práctica sin grabar primero** para recordar curvas, paradas, puntos de cambio. Después se graba la versión limpia.

Dispositivo de entrada y fidelidad del steering grabado

Detalle importante para quien implemente el patrón: `Car.GetSteering()` devuelve **el input crudo del dispositivo activo en ese momento**, no una representación abstracta del ángulo del volante in-game. Esto significa que la calidad del steering grabado depende del hardware del que grabó:

Dispositivo	Valores grabados
Teclado (A/D)	Discreto: -1, 0, +1
Controller (Xbox/PS, stick analógico)	Continuo: cualquier float -1..+1
Volante (Logitech, Thrustmaster, etc.)	Continuo, alta resolución

En el caso del proyecto `BrigadaZ_Transport`, la grabación se hizo con teclado. El CSV resultante tiene `steering` casi siempre en 0, con spikes ocasionales de -1 o +1. Esos spikes no son data útil para playback (causarían volantazos discretos en vez de steering suave), así que se decidió **descartar el steering grabado y usar pure pursuit calculado**.

Si la grabación se hubiera hecho con un volante físico, el CSV tendría valores continuos representando la compensación humana real (incluyendo micro-correcciones por drift natural del vehículo). En ese caso convendría usar el steering grabado como source-of-truth para playback (sumándolo o sobrescribiendo el pure pursuit). Para el manual: documentar esta bifurcación de pipeline y dejar el switch como configurable (`useRecordedSteering: true/false`).

Para la comunidad de modding de DayZ (mayormente teclado), pure pursuit calculado va a ser siempre la elección estándar.

Por qué esta sección importa para otros modders

El equilibrio que articulamos durante el desarrollo: "no pretendemos que el bus se comporte como humano, pero podemos encontrar un punto medio donde nosotros grabemos ayudándolo y él reproduzca lo mejor que puede". Esta es la mentalidad correcta para colaboración humano-IA en general, no solo para este framework. Vale la pena explicitarlo en el manual porque muchos intentan grabar como manejarían normalmente y se frustran con el playback.

Decisión de diseño: el framework NO esconde errores de grabación

Durante el desarrollo nos topamos varias veces con la tentación de agregar lógica defensiva al framework que "corrigiera silenciosamente" una grabación sub-óptima. Por ejemplo: si el operador grabó parando en 4ta marcha, el bus en el playback intenta arrancar en 4ta y no tiene torque suficiente para moverse desde 0 km/h.

La tentación es: agregar código que detecte "bus parado, gear alto" y fuerce un downshift automático. Eso haría que el bus arranque sin problema, sin pedirle al operador que corrija nada.

Decidimos no hacerlo, y la razón es estructural: si el framework oculta los errores de grabación, el operador no recibe feedback y va a seguir grabando mal sin saberlo. Su próximo recording va a tener el mismo problema en algún punto donde la lógica defensiva no llega, y ahí va a fallar de manera más confusa (porque ya no se entiende qué causó el problema — el framework "a veces" lo soluciona y "a veces" no).

Es preferible que el framework sea **honesto y predecible**: reproduce lo que recibe. Si recibe una grabación con gear=5 y velocidad=0, el bus se queda trabado intentando arrancar en 5ta. El operador ve el problema, vuelve a grabar bajando marchas progresivamente, y el siguiente playback funciona.

Este criterio aplica a cualquier framework que tome input humano para reproducirlo: agregar capas de "smart correction" parece amable pero es contraproducente cuando el sistema se usa repetidamente — el usuario nunca aprende a hacerlo bien.

Reglas claras > comportamiento mágico.

Capítulo 6 — Casos de uso vehiculares

Este capítulo presenta casos de uso concretos que el framework habilita. La lista no es exhaustiva — es un menú para inspirar a otros modders y demostrar que eAI driving deja de ser "un bus" para convertirse en una categoría amplia de contenido posible.

Las ideas a continuación surgen pensando en un server **PVE** (como el nuestro), pero la mayoría se adaptan a PVP cambiando solo las facciones y la hostilidad de los vehículos AI.

6.1 Transporte público

El caso canónico — el bus que demuestra el framework en este proyecto. Vehículo recorre rutas regulares con paradas configurables, los jugadores lo usan como medio de transporte real, la UI les muestra ETA y próximas paradas.

Variantes: taxi por demanda (jugador llama desde smartphone), servicio nocturno con paradas distintas, líneas express vs locales.

6.2 Transporte marítimo

Servicio de ferry / lancha entre puntos costeros o islas. En agua hay **casi cero obstáculos físicos**, lo que hace el pathfinding mucho más simple que en tierra — solo hay que evitar la línea de costa.

Casos específicos:

- **Excursiones a islas** — el vehículo AI es el único transporte disponible a ciertas zonas (jugadores no pueden o no quieren llegar por su cuenta)
- **Acceso exclusivo a zona prohibida** — combinando con un `CylinderTrigger` que bloquea otros vehículos, el bote AI se vuelve el ÚNICO modo de cruzar a una isla restringida. Mecánica única en el server
- **Contrabando** — bote AI con cargamento ilegal que el jugador puede

interceptar o escoltar dependiendo del rol

Pendiente técnico: adapter de BoatScript (Capítulo 3.4 — v1.2).

6.3 Servicios de logística automatizada

Cambia el modelo de juego de "el jugador va al trader" a "el servicio va al jugador".

Ambulancia para emergencias Sistema donde el jugador con baja vida puede llamar una ambulancia (via smartphone, radio, o item específico). Vehículo AI conducido por un médico-bot llega al punto, entrega medicamentos / kit de primeros auxilios, se va. Reemplaza el "viaje de vuelta a la base" después de un evento médico.

Delivery de armas estilo airdrop pero controlado Para misiones, en vez de aparecer un drop random, un vehículo AI lleva el equipamiento al jugador. Más realista, más cinematográfico, evita que el jugador vuelva a base/trader a rearmarse cuando está en operación.

Servicio de entregas a domicilio Tu Amazon Prime de DayZ. El jugador hace un pedido desde una UI ligada a un objeto (PC, smartphone, radio), elige items (armas, ropa, equipamiento, materiales de construcción, según qué venda cada server), y un vehículo AI lo lleva a la coordenada del jugador. Cambia el meta: los jugadores no tienen que abandonar bases o operaciones para abastecerse. Habilita "comercio en territorio enemigo" en PVP.

6.4 Patrullas y escoltas

Patrullas militares motorizadas Convoy de 1-3 vehículos AI con faction hostil recorriendo rutas estratégicas (rutas a bases, perímetros, accesos a ciudades). Atacan si detectan jugador / amenaza. Reemplazan los "spawns aleatorios de enemigos" con presencia continua y reactiva.

Patrullas costeras / marítimas Idem pero por agua. Mucho menos común en DayZ que las patrullas terrestres, abriría dinámicas nuevas (jugadores teniendo que evitar la línea de costa, contrabandistas escondiendo botes).

Convoys con escolta Vehículo central (trader, líder, VIP) con vehículos AI escolta adelante y atrás. Si el jugador ataca, los escoltas responden.

6.5 Misiones con vehículos AI como elemento central

Carreras humano vs IA "Ganale a Boris al punto X." El bus / vehículo AI hace su ruta normal, el jugador tiene que llegar antes a un destino marcado. Simple, divertido, showcase del framework para nuevos jugadores del server.

Persecuciones / fugas

- "Persigue y elimina a Boris y acompañantes antes de que lleguen a X." El vehículo AI huye hacia un punto seguro; el jugador tiene una ventana para interceptarlo
- Inverso: vehículo AI hostil persigue al jugador. Patrulla policial o mafia (pendiente investigación: AI persecution behavior)

Mission narrativa: "Mysterious Boris" Mecánica multi-paso que usa eAI driving como medio para una experiencia única:

1. Jugador se sube al vehículo de un NPC misterioso (Boris)
2. UI overlay tapa la visión del jugador (simula venda en los ojos),
bloquea el mapa
3. Vehículo AI conduce a una zona oculta
4. Al llegar, visión vuelve pero el mapa sigue bloqueado. Zona
delimitada por `CylinderTrigger` de rebote (si el jugador intenta salir caminando, es repuesto
adentro)
5. Jugador completa misión en la zona
6. Vuelve al vehículo, le tapan visión nuevamente
7. Boris lo devuelve al punto inicial, visión y mapa se restauran

Bonus: **el vehículo de Boris es el único que puede entrar a esa zona** (por el `CylinderTrigger`). Otro jugador amigo podría intentar seguirlos para descubrir la ubicación, pero estaría limitado por el mismo trigger — sabe aproximadamente la dirección pero no la zona exacta, ni con helicóptero.

Esto demuestra que eAI driving + `CylinderTrigger` + UI overlay = una categoría de **gameplay narrativo nuevo** que antes requería que el NPC fuera otro jugador humano.

Misiones encadenadas con acciones de bot "Acompaña a Boris al trader de drogas y evitá que lo maten en el camino" — durante el viaje, vehículos AI hostiles intentan interceptar. Cuando llega, Boris puede salir del vehículo y hacer una acción (disparar, hablar con NPC, dejar paquete), pedir refuerzos, etc. El viaje vehicular es solo una pieza de una secuencia más larga.

6.5b "Ladrones de vehículos" — bots AI con objetivo de robo

Caso muy interesante que cambia la dinámica del juego para jugadores que tienden a dejar vehículos abandonados en cualquier lado. Bots AI dispersos por el mapa (en grupo o solos, sin armas, sin conducta agresiva contra el jugador, pero con buena capacidad de correr) que tienen como objetivo:

1. Patrullar su radio buscando vehículos sin conductor
2. Si encuentran uno **en la calle** (no en zonas de bases / safe
zones), tienen capacidad de **forzarlo / desbloquearlo**
3. Si el jugador está al volante pero a puerta abierta o sin
trabar, el bot puede sacarlo del asiento
4. Se lleva el vehículo a un "**desarmadero**" (zona predefinida)
5. Variante hard: al llegar al desarmadero, el vehículo se borra
(loot permanente perdido)
6. Variante intermedia: el jugador puede ir al desarmadero a
recuperarlo, o interceptar en el camino

Posibilidades de extensión:

- Combinar con patrulla armada esperando afuera del desarmadero
(el jugador que intenta recuperar tiene que pasar el filtro)

- Reward para el jugador que mata al bot ladrón antes de que el vehículo llegue al desarmadero
- Sistema de "rescate" donde el jugador puede pagar para liberar el vehículo del desarmadero (mecánica económica adicional)

Esta categoría agrega **consecuencia al gameplay** de quien no cuida sus vehículos, sin necesidad de PvP forzado. Casa bien con servers PVE que quieren agregar tensión sin combate humano contra humano.

6.6 Casos pendientes de investigación

Algunas ideas requieren capacidades que todavía no validamos:

- **Persecuciones AI activas** (no solo "ir a punto X" sino "perseguir al jugador adaptativamente")
- **Combate desde vehículo en movimiento** (bot saca arma, dispara por ventana mientras conduce o desde asiento de pasajero)
- **Coordinación entre múltiples vehículos AI** en tiempo real (rebasarse, mantener formación, dispersarse ante amenaza)
- **Helicópteros** (Capítulo 3.4 — v2.0+)

6.7 PVE vs PVP

Las ideas anteriores están pensadas desde un server PVE como el nuestro, donde el AI conviene que sea no-hostil o cooperativo. La mayoría se adapta a PVP cambiando solo:

- **Facción** del vehículo AI (de Civilian / Friendly a Hostile)
- **Comportamiento ante jugadores** (en PVP, atacar; en PVE, ignorar o asistir)
- **Drops** del NPC al morir (en PVP, el botín es parte del incentivo)

La mecánica subyacente del framework no cambia. Es solo configuración de facciones de eAI, que ya es estándar en Expansion-AI.

Capítulo 7 — Casos de uso no vehiculares

7.1 Sistema "Ring the bell" — paradas inteligentes (futuro v2 del Transport)

(no implementado en v1.0)

En v1.0 el bus para en cada parada sin excepción, como un colectivo turístico. Para v2 se planeó un sistema más sofisticado donde:

- **Jugador onboard** puede señalar en qué parada se baja (botón en la UI del pasajero, o tirando del "cordel" del bus, o pulsando un icono flotante de "next stop")
- **Jugador en la parada** levanta la mano (acción de teclado o emote) para indicar que quiere subirse
- El bus chequea estas señales en cada parada y decide si frena o sigue de largo
- Optimiza tiempo total del viaje si el bus va vacío y nadie espera

Funciones `HasWaitingPlayers` y `HasOnboardPassengers` quedaron en el código `BZBusService` para reutilizarse cuando se implemente. La lógica de v2 sería: en lugar de chequear "¿hay alguien?", chequear "¿alguien señaló que quiere subir/bajar?".

Por qué se descartó del v1.0: el scan en cada parada era único en el momento de llegada (a 15m del cartel). Si un jugador llegaba a la parada mientras el bus ya había pasado el scan, el bus seguía de largo aunque estuviera literalmente al lado. Para la versión inicial es mejor el comportamiento simple y predecible (siempre para) que uno "inteligente" con timing problemático.

Apéndice A — Disclosure de metodología (uso de IA)

(pendiente — texto formal para el paper explicando nuestro rol como autores y el del asistente IA. Ver lineamientos acordados en la metodología del Apéndice E.)

Observación a incorporar — IA enseñando a IA *(reflexión 2026-05-22, articulada por Sonom4n durante la implementación del AI logging):*

El framework está mediado por **dos sistemas de IA distintos operando en planos temporales separados**:

- **LLM (Claude)** en tiempo de **desarrollo**: razona sobre física, control, código. Lee logs, propone hipótesis, escribe el código del framework. La eAI nunca ve ni entiende este código — solo lo ejecuta.
- **eAI (Expansion-AI)** en tiempo de **runtime**: ejecuta los comandos de bajo nivel del vehículo (`SetThrottle` , `ShiftTo` , `SetSteering`). No tiene noción del recording ni del control predictivo — solo recibe inputs cada frame.

Entre ambos hay **tres capas de traducción de intención**:

1. **Operador humano** → tiene la intención (qué quiere que pase)
2. **Grabación (recording)** → captura inputs y trayectoria
3. **Capa de adaptación escrita por el LLM** → traduce el recording a comportamiento ejecutable por la eAI considerando física, calibración del vehículo, condiciones ambientales
4. **eAI** → ejecuta los comandos resultantes

La eAI por sí sola no podría manejar un bus por una ruta costera con paradas exactas — falta toda la lógica de control predictivo, modos parking/crucero, inferencia de gear, detección por distancia. Esa lógica salió de una conversación iterativa entre el operador y el LLM, materializada como código que la eAI consume sin entenderlo.

Para el paper: es uno de los pocos casos documentados donde un LLM actúa como **capa de traducción entre intención humana y comportamiento de AI scripted**. La metodología del trabajo (recording-driven development con LLM como ingeniero de control) puede ser tan transferible como el framework mismo.

Vale como párrafo del apéndice A o como sección distinta sobre "Metodología del desarrollo asistido por IA en proyectos de modding".

Refinamiento técnico — hijacking de un sistema autónomo (*refinamiento de Sonom4n, 2026-05-22*):

No modificamos el código interno de eAI. No reescribimos su lógica de decisión. Solo nos comunicamos con eAI a través de su **API pública** de inputs:

- `Car.SetThrottle(value)`
- `Car.SetBrake(value)`
- `Car.SetSteering(value)`
- `Car.ShiftTo(gear)`

eAI fue diseñada para decidir sola (pathfinding interno, target acquisition, movement planning). El framework la **convierte en una capa de ejecución sin decisión**, alimentándole inputs precalculados desde nuestra propia lógica externa. eAI nunca sabe que está siendo "secuestrada" — solo recibe valores y los aplica.

En términos arquitectónicos esto es un **adapter pattern para AI scripted**: el framework actúa como adapter entre el dominio de alto nivel (recording + intención + condiciones) y la superficie de API de bajo nivel de la AI cerrada.

Implicancias del patrón:

- Es **transferible** a cualquier sistema de AI con superficie de API conocida, no solo a eAI. Cualquier mod con AI scripted (vehicle AI, combat AI, navigation AI) puede ser "hijackeado" de esta forma.
- Es **robusto a actualizaciones del sistema controlado**: si eAI cambia su lógica interna en una nueva versión, el framework sigue funcionando mientras la API de inputs se mantenga.
- No requiere **fuentes abiertas del sistema controlado**: incluso con eAI binarizada y sin acceso al código, basta con saber qué métodos públicos exponen las clases base.

Esto es un caso de estudio fuerte para argumentar que el desarrollo asistido por LLM puede operar sobre sistemas cerrados sin necesitar acceso a su implementación — solo a su superficie observable.

Análisis lateral de trayectoria — 8 wp problemáticos identificados (*análisis 2026-05-22, 3 tomas IA vs recording humano*):

Después de cerrar el análisis de velocidad (que validó el trade-off ya elegido — ver siguiente sección), corrimos un segundo análisis enfocado en **desviación lateral de trayectoria** (distancia del bus IA al

punto más cercano del trazo humano). Los resultados:

- **Desviación promedio por tramo:** 0.88 - 1.16m (dentro del ancho propio del bus — bien)
- **P95 (no-outliers):** 1.90 - 2.38m (aceptable, dentro de la calzada)
- **Tramos completos:** <1% de muestras se desvían >3m

Pero al filtrar **wp_idx** donde las 3 tomas se desvían >2m de forma consistente (no ruido aleatorio), encontramos **8 puntos específicos con divergencia sistemática**:

Zona	wp_idx	Pos	Desv. avg	Diagnóstico
Zigzag Kamenka→Komarovo	543	(2809, 2019)	3.93m	El peor de todos
Zigzag mismo tramo	605	(3091, 2112)	2.22m	Mismo cluster
Rotonda Chernob entrada	1557	(6210, 2366)	2.18m	Pre-Chernogorsk
Rotonda Chernob curva	1574	(6214, 2394)	2.32m	Cluster rotonda
Rotonda Chernob salida	1577	(6214, 2399)	2.02m	Cluster rotonda
Salida del spawn	24, 26	(1150, 2387)	~2.4m	Inicial, ignorable
Approach Kamenka	182	(1696, 2204)	2.36m	Pre-stop

Los dos clusters principales (zigzag y rotonda Chernob) coinciden con las zonas que el operador percibía como "delicadas" durante el manejo ingame. La data confirma su intuición.

Causa técnica: el `LOOKAHEAD_DIST = 10.0` (pure pursuit) era **fijo**. En rectas funciona bien. En curvas pronunciadas apunta a un punto que ya está "del otro lado" de la curva → el bus corta por adentro o se va por afuera. Cuanto más cerrada la curva, peor el efecto.

Solución — Lookahead adaptativo (v1.2, build 2026-05-22 23:41):

```
private float ComputeLocalCurvature() {
    // Suma cambios absolutos de heading en proximos N waypoints (rad)
    // Recta: ~0 rad. Curva fuerte: 1.0-2.0+ rad.
}

private float ComputeAdaptiveLookahead() {
    float curvature = ComputeLocalCurvature();
    if (curvature <= LOOKAHEAD_CURVATURE_LOW) return LOOKAHEAD_DIST; // 10m
    if (curvature >= LOOKAHEAD_CURVATURE_HIGH) return LOOKAHEAD_DIST_MIN; // 5m
    // Interpolacion lineal
    float factor = (curvature - LOOKAHEAD_CURVATURE_LOW) / (LOOKAHEAD_CURVATURE_HIGH - LOOKAHEAD_CURVATURE_LOW);
    return LOOKAHEAD_DIST - factor * (LOOKAHEAD_DIST - LOOKAHEAD_DIST_MIN);
}
```

Propiedades del fix:

- ✓ NO toca velocidad (no rompe el trade-off precisión)
- ✓ NO toca frenado (no afecta llegada exacta a paradas)
- ✓ Solo afecta steering en curvas → mejora geometría sin sacrificios
- ✓ Adaptativo: en rectas el bus sigue con lookahead largo (suavidad);

en curvas se acorta automáticamente (precisión)

Validación: pendiente toma 4 con el fix activo. Comparar desviación en los 8 wp identificados — si bajan de 2-4m a <1.5m, fix exitoso.

Para el paper: este es el ejemplo más claro de la metodología "recording-driven development con LLM como ingeniero de control":

1. Grabar 3 corridas de la IA (PathLogger AI mode)
2. Analizar desviación vs recording humano
3. Identificar puntos sistemáticos (no ruido)
4. Hipotetizar causa técnica (LOOKAHEAD_DIST fijo)
5. Diseñar fix que NO viole los compromisos arquitectónicos
6. Validar con nueva toma

Los datos guían la corrección. La corrección es quirúrgica, no especulativa. **Es exactamente el ILC manual** que describimos en secciones anteriores — solo que el LLM toma el rol del controlador adaptativo en lugar de ser código autónomo.

Reformulación arquitectónica — System Identification de eAI (*reflexión de Sonom4n 2026-05-23 mañana, después de leer el paper*):

Después de la sesión del 22 (corrección tramo por tramo + análisis de trayectoria), el usuario propuso un reframe fundamental del approach:

Esto es **exactamente System Identification** en teoría de control: en vez de ajustar el controlador ante cada desviación observada, caracterizá la **función de transferencia interna** del sistema controlado. Una vez conocida, podés calcular cualquier input necesario para conseguir cualquier output.

Evidencia empírica del patrón (ya teníamos):

- AvgThrottle IA 0.71 vs Humano 0.61 → la IA pisa MÁS pedal de acelerador
- MaxSpeed IA 70 vs Humano 80 km/h → la IA consigue MENOS velocidad

Si eAI fuera lineal y transparente, más throttle daría más velocidad. **No pasa.** La discrepancia solo se explica si eAI hace algo internamente que atenúa nuestros inputs en ciertos contextos. **Hay un patrón, y es medible.**

Hipótesis sobre la forma del patrón (a verificar empíricamente):

Hipótesis	Qué hace eAI	Cómo se detecta
Filtro low-pass de inputs	Suaviza cambios bruscos	Step input, medir tiempo de respuesta
Cap de velocidad por curvatura	Reduce throttle en curvas	Throttle=1 en curva vs recta

Hipótesis	Qué hace eAI	Cómo se detecta
Lógica anti-collision interna	Sobrescribe inputs cerca de obstáculos	Con/sin obstáculos próximos
Curva de respuesta no-lineal	<code>throttle=1.0</code> $\neq 2 \times$ <code>throttle=0.5</code>	Varios valores fijos, medir aceleración
Estado acumulado	Pondera inputs previos	Mismo input en distintos contextos

Implicancia para el roadmap del framework:

La "Toma 2 (calibración vehicular)" estaba pensada para medir parámetros físicos del vehículo (MAX_BRAKE_DECEL real, etc.). La reformulación de Sonom4n la **amplía**:

- **Toma 2.A:** caracterización física del vehículo (lo que teníamos)
- **Toma 2.B:** caracterización de la función de transferencia de eAI

(lo nuevo) — experimentos diseñados para revelar el patrón interno del controlador

Si tenemos ambas caracterizaciones, el playback puede compensar **pre-emptivamente** cualquier patrón de eAI antes de que afecte la ejecución. No corregimos consecuencias — pre-compensamos causas.

Conexión con el principio "no peleamos contra eAI" (formulado el día anterior):

La caracterización del patrón es **respetar al sistema cerrado en su máxima expresión**: entendemos cómo procesa nuestros inputs, no intentamos cambiar cómo lo hace internamente. Es la culminación del principio adapter — la fidelidad última a la API de eAI.

Caracterización física del bus — System Identification empírica completa (experimentos 2026-05-23, 3 terrenos):

Para caracterizar el comportamiento físico del bus (separado del comportamiento de eAI, que ya está caracterizado en sección siguiente), corrimos step response en tres condiciones:

Terreno	Setup	Resultado
NWAF runway (plano)	1 km asfalto recto	Baseline: 0 → 55 km/h en 13s
Sonomir → Solnichniy (subida 4.43°)	503m, +39m elevación	0 → 45.9 km/h en 13s (-9.1)
Sonomir → Solnichniy (bajada 4-7° variable)	Misma carretera, dirección inversa	0 → 71.8 km/h en 13s (+16.8)

Aceleración por gear en cada terreno

Gear	Plano (m/s ²)	Subida (m/s ²)	Bajada (m/s ²)	Δ subida vs plano	Δ bajada vs plano
2 (FIRST)	2.12	2.05	2.77	-0.07	+0.65
3 (SECOND)	1.54	0.80	2.71	-0.74	+1.17
4 (THIRD)	~1.1	—	1.42	—	+0.32

Gear	Plano (m/s ²)	Subida (m/s ²)	Bajada (m/s ²)	Δ subida vs plano	Δ bajada vs plano
5 (FOURTH)	~0.5	—	1.20	—	+0.70

Los datos de gear 3 son los más limpios (gear 2 está afectado por la zona plana de inicio de la pendiente; gear 4-5 son menos puntos).

Validación del modelo físico simple

Hipótesis: la aceleración del bus en pendiente sigue la fórmula elemental de cinemática:

$$a_{\text{efectiva}} = a_{\text{motor_plano}} \pm g \times \sin(\theta_{\text{pendiente}})$$

donde **+** es asistencia gravitatoria (bajada) y **-** es resistencia (subida).

Comparación predicción vs medición (gear 3, pendiente promedio 4.43°):

Predicción	Medición	Discrepancia
$g \cdot \sin(4.43^\circ) = 0.76 \text{ m/s}^2$	-0.74 m/s ² (subida)	<3% error
$g \cdot \sin(4.43^\circ) = 0.76 \text{ m/s}^2$	+0.70 m/s ² (bajada, en zona final donde pendiente es ~4.4°)	8% error

Hallazgo importante — pendiente no uniforme: análisis de velocidad vertical / horizontal mostró que la carretera Sonomir es **más empujada al tope (~6.7°)** y **más suave al pie (~4.2°)**. En la bajada gear 3, el bus venía con pendiente local ~6° y la aceleración medida (+1.17 m/s²) coincide con $g \cdot \sin(6^\circ) = 1.02 \text{ m/s}^2$ dentro del 12% de error.

Conclusión

El modelo físico simple predice los efectos de pendiente con <10% de error cuando se conoce la pendiente local. Suficiente para implementación práctica.

Implicancia para el control predictivo

La fórmula actual del freno predictivo no considera pendiente:

```
// Actual (BZBusService.c)
float u_ms = kmh / 3.6;
float aNeeded = (u_ms * u_ms) / (2.0 * distRemaining);
float brakeFrac = aNeeded / MAX_BRAKE_DECEL;
```

Con factor pendiente sería:

```
// Propuesto
float pendiente = (waypoint.Y - bus.Y) / horizontalDist; // signed
```

```
float gAssist = 9.8 * pendiente; // positivo en bajada, negativo en subida
float aNeeded = (u_ms * u_ms) / (2.0 * distRemaining) - gAssist;
// En bajada: aNeeded mayor (más freno necesario)
// En subida: aNeeded menor (menos freno, la gravedad ayuda)
float brakeFrac = aNeeded / MAX_BRAKE_DECEL;
```

Implementación: agregar `ComputeLocalPendiente()` en `BZBusService` que mira los próximos 5 waypoints adelante, calcula pendiente promedio, y la usa para ajustar `aNeeded`. ~10 líneas de código.

Implicancia para el paper

Tres puntos fuertes:

1. **Validamos un modelo físico simple** (cinemática elemental) con experimentación empírica en tres terrenos. Encaje <10% es publicable.
1. ****Distinguimos limitaciones del bus físico vs limitaciones del adapter** vs limitaciones de eAI**. Cada uno está caracterizado independientemente.
1. **El framework es ahora "physics-aware"**: con tres datasets podemos inferir parámetros del vehículo (drag, max accel, max brake) y del entorno (pendiente). Eso permite calibración automática de cualquier vehículo en cualquier ruta, no solo el bus en Chernarus.

Qué dicen estos datos sobre la precisión de eAI

(respuesta a una pregunta directa de Sonom4n 2026-05-23, vale como sección del paper)

Estos experimentos NO miden la precisión de eAI directamente. Miden la precisión del **bus físico + nuestro adapter** después del hijacking descrito en sección anterior. Durante los step responses, eAI calculó sus inputs (cap 50 km/h, gear FIRST, fórmula naive), pero los sobrescribimos en cada frame con throttle=1, brake=0, steering=0.

Si eAI hubiera manejado SIN nuestro override, los terrenos darían:

Terreno	eAI sola (predicción de su fórmula)	Realidad con adapter
Plano	Satura a 50 km/h (<code>speedCoef=0</code>), gear FIRST permanente	76 km/h, gear 5
Subida 4.43°	Probablemente <30 km/h (gravedad opone + cap interno)	45.9 km/h, gear 3
Bajada	Descontrolada — sin brake adaptativo, sin predicción de pendiente	71.8 km/h controlados

Conclusión técnica fuerte: la precisión que medimos NO es atribuible a eAI. Es del:

- Engine de DayZ (motor + drag + gravedad simulada — fidelidad física razonable)
- Nuestro adapter (cálculos predictivos que sobrescriben a eAI)
- Recording humano (define la intención que el adapter ejecuta)

eAI por sí solo, según su código (ver fórmula en sección siguiente), es inadecuado para tareas que requieran:

- Velocidades sostenidas >50 km/h
- Respuesta predictiva a pendiente
- Frenado adaptativo

Confirmación de la asimetría técnica de eAI

Estos experimentos refuerzan empíricamente lo que la lectura del código fuente ya sugería: **eAI tiene dos personalidades técnicas distintas.**

Módulo	Estado
Combate	Sofisticado: ballistics, lead targets, recoil compensation, pathfinding refinado. Precisión sub-segundo a 300m (observación empírica clásica).
Vehicular	Naive: 4 coeficientes lineales + gear FIRST hardcoded. Sin nuestro adapter, ~inutilizable para precisión.

La asimetría no es por capacidad teórica de eAI — es por **nivel de desarrollo del módulo**. Probablemente el módulo vehicular es un primer pase no optimizado, el de combate es producto de años de iteración.

Para el paper, esto justifica la tesis del framework: el approach de adapter + override + control predictivo es **necesario** porque eAI vehicular no es suficiente. No estamos compitiendo con eAI — estamos **rellenando una capacidad faltante** mediante hijacking selectivo.

Es el equivalente técnico de: "el módulo X de este sistema es naive, pero su superficie de API permite controlarlo desde afuera, así que lo trato como un actuador low-level y le pongo encima la inteligencia que le falta".

Caracterización de eAI vehicular — fórmula completa descubierta *(investigación 2026-05-23, código fuente extraído de ai_scripts.pbo):*

Después del experimento curve (NUMPAD 9) que llegó a 76.1 km/h en plano, investigamos el código fuente de eAI extrayendo el PBO con BankRev. **Encontramos la fórmula exacta del driving de eAI en entities/carscript.c :**

```
// eAI vehicle driving (resumido):
override void OnInput(float dt) {
    eAIBase driver = CrewMember(VEHICLESEAT_DRIVER);
    if (!driver) return;

    vector wayPoint = driver.m_PathFinding.GetNext(...);

    float turnCoef      = max(dot(direction, pathDir), 0.1);
    float speedCoef     = LinearConv(0, 50, currentSpeed, 1, 0); // CAP 50 km/h
    float distanceCoef = LinearConv(0, 50, distanceToWp, 0, 1);
```

```
float rpmCoef      = LinearConv(0, redline, currentRPM, 1, 0);

float throttle = turnCoef * speedCoef * distanceCoef * rpmCoef;
float brake     = (1 - speedCoef) * (1 - distanceCoef);

SetSteering(steering);
SetThrottle(throttle);
SetBrake(brake);
ShiftTo(FIRST); // siempre FIRST, nunca shifta a SECOND+
}
```

Tres hallazgos críticos:

- 1. **Cap de velocidad de 50 km/h hardcodedo** en `speedCoef` . A 50 km/h ese coeficiente llega a 0, multiplica todo → throttle 0. **Por eso los AI driving sin adapter "saturan" a 50 km/h.**
- 1. **eAI siempre maneja en gear FIRST**. Nunca usa SECOND/THIRD/etc. Por eso un bus manejado por eAI sin override es lentísimo y va siempre rugiendo el motor a alto RPM.
- 1. **La "inteligencia" de eAI vehicular es naïve**: 4 coeficientes lineales y un gear hardcodedo. Comparado con la sofisticación de eAI en combate (ballistics, lead, recoil compensation), el módulo vehicular **está claramente subdesarrollado**.

Nuestro adapter neutraliza completamente eAI:

En `BZBusCarScript.OnInput` , primero llamamos `super.OnInput(dt)` (que ejecuta el OnInput de eAI con su cap 50 km/h), DESPUÉS sobrescribimos con `srv.ApplyBusInput(this, dt)` . Como los inputs se aplican una vez por frame, **gana el último valor escrito = el nuestro**.

Capa	Comportamiento	Efecto en motor
eAI driving	cap 50 km/h, FIRST gear, pathfinding básico	Anulado en cada frame
Nuestro adapter	inputs calculados por control predictivo	Aplicado al motor

Validación empírica: el curve test alcanzó **76.1 km/h** en plano. Si el cap de eAI estuviera activo, hubiera saturado a 50. **No saturó**. Confirmación de que el adapter sobrescribe correctamente.

Implicaciones para el paper:

- 1. **El "patrón de eAI" buscado YA está caracterizado completamente**
(ya no necesitamos System Identification para eAI, sí para el engine de DayZ + drag aerodinámico).
- 1. **Hijacking exitoso documentado:** el adapter pattern + override de OnInput es la técnica para neutralizar un sistema cerrado sin tocar su código.
- 1. **Refuerza la tesis del paper:** el framework no compite con eAI, lo reemplaza tácticamente sin modificar su código. Es **architecture by subordination** — eAI sigue ejecutándose pero sus outputs se ignoran selectivamente.

1. **Asimetría de precisión explicada:** combate vs vehículos no es por

capacidad de eAI sino por nivel de desarrollo del módulo. La precisión de combate sale de años de iteración; vehicle driving es probablemente código de un primer pase no optimizado.

Para el manual: si alguien quiere implementar AI driving en otro mod, debería:

- **NO usar eAI vehicular nativo** (cap 50 km/h, FIRST gear, naive)
- **SÍ usar eAIBase como driver "shell"** y reemplazar el OnInput del CarScript con su propia lógica de adapter
- Aprovechar `m_PathFinding` del driver eAI si necesita navegación reactiva (a obstáculos), o usar waypoints precomputados como nosotros

Este patrón es **transferible a cualquier sistema cerrado**: mod de AI scripted, frameworks de IA en general, sistemas legacy con comportamiento subóptimo que se quiere mejorar sin tocar el código.

La asimetría de precisión de eAI (*observación de Sonom4n 2026-05-23 mientras esperaba server*):

Esta es una observación **fundamental** que cambia el framing de los experimentos de System ID. eAI usado para combate (apuntar, disparar) es altamente preciso — los bots a pie son letales a cientos de metros. Pero eAI usado para vehículos parece "torpe" comparativamente.

Hipótesis técnica: eAI ofrece **dos clases de API** según el dominio:

Para combate	Para vehículos
Alto nivel ("apuntá a este target")	Bajo nivel (<code>SetThrottle</code> , <code>SetBrake</code> , <code>SetSteering</code>)
eAI maneja angles, recoil, ballistics, timing	eAI solo ejecuta lo que le decimos
Le decís el QUÉ, eAI calcula el CÓMO	Nosotros calculamos el cómo, eAI solo passthrough

Si esta hipótesis es correcta, **nuestros experimentos de System ID caracterizan el engine vanilla de DayZ, NO a eAI**. La "precisión que falta" no es de eAI — es nuestra al calcular los inputs a bajo nivel. La eAI no aplica nada inteligente sobre nuestros throttle/brake, solo los pasa al motor del juego.

Implicación para el paper: el approach que tenemos ahora (adapter sobre inputs raw) **no aprovecha la inteligencia que eAI demuestra en otros dominios**. Hay (probablemente) una API de alto nivel para vehículos que sí la aprovecha — algo como `Vehicle.DriveTo(position)` o `Vehicle.SetTargetSpeed(kmh)` . Si existe y la usamos, el bus podría ser tan preciso conduciendo como los bots de combate apuntando.

Investigación pendiente:

- ¿Qué APIs de alto nivel ofrece eAI para vehicle driving?
(`eAIBase.SetVehicleDestination` ? métodos similares?)
- ¿Cómo es la curva de delegación? Más delegación = más precisión pero menos control sobre el "estilo de manejo" (que es justo lo que el framework quiere preservar como "intención del operador").
- Trade-off: si delegamos a eAI, perdemos el control fino que graba el

PathLogger. El recording deja de ser "lo que vos manejaste" y se vuelve "tu intención abstraída".

Para el paper como discusión: tenemos dos approachs ortogonales:

1. **Approach actual (passthrough):** controlamos cada input, eAI ejecuta.

Pros: fidelidad al recording humano. Contras: heredamos limitaciones del engine DayZ.

2. **Approach delegado (high-level):** le decimos a eAI "andá a X con

velocidad Y", eAI maneja todo internamente. Pros: aprovecha la inteligencia de eAI. Contras: perdemos fidelidad al estilo del operador, eAI puede manejar "a su modo".

El framework podría ofrecer **ambos** como modos configurables. Esa es otra dimensión del CONSERVATIVE vs FAITHFUL que ya teníamos en roadmap.

Pistas custom para experimentos controlados (*propuesta de Sonom4n 2026-05-23 al salir*):

Para los experimentos de System Identification necesitamos rectas largas en distintas condiciones (plano, subida, bajada con pendiente específica). **Chernarus no siempre ofrece esas condiciones puras** — las pendientes suelen estar en zonas montañosas con curvas, y las rectas largas naturales (aeropuertos) son casi todas planas.

Idea: usar **DayZ Editor Loader** para construir **pistas artificiales de asfalto** con pendiente y largo controlados:

- Pista plana de 1km: baseline
- Pista en subida ~5%, 400m: medir resistencia gravitatoria
- Pista en bajada ~5%, 400m: medir asistencia gravitatoria
- Pistas con pendientes pronunciadas (~10%) para puntos extremos

Ventajas sobre rectas naturales:

- Control absoluto de las variables (pendiente, largo, superficie)
- Sin obstáculos contaminantes
- Reproducible: cualquier modder puede recrear la misma pista usando el mismo .dze file

Ventajas para el paper:

- Hace los experimentos de System ID **reproducibles** por terceros
- Convierte el método de "experimento ad-hoc en el mapa" a "protocolo estandarizado con infraestructura propia"
- Las pistas custom pueden distribuirse junto al mod como "test infrastructure" del framework

Para el roadmap: incluir las pistas .dze de testing como parte del release del framework. Modder que quiera caracterizar otro vehículo o otra AI scripted, descarga las pistas, hace los experimentos, obtiene sus parámetros de calibración.

Reflexión más profunda — eAI en pistas custom:

La imagen es lúdica pero apunta a algo serio: convertir a eAI en **sujeto experimental** dentro de un laboratorio que nosotros mismos construimos. Es exactamente el paradigma de **test fixture en ingeniería de control**: para caracterizar un sistema, lo aislás en condiciones conocidas y le medís la respuesta. Lo que en hardware se hace con dynos y rolling roads, acá lo hacemos con pistas .dze en DayZ.

Extensión — infraestructura real de gameplay (*idea de Sonom4n también 2026-05-23*):

Las pistas custom no son solo para experimentos. Si un admin de server quisiera **diseñar una autopista** (o un túnel, o un puente, o una carretera intercity inexistente en Chernarus vanilla), nuestro framework puede correr buses ahí sin modificación. El framework es agnóstico al mapa — solo le importa la geometría de la ruta grabada.

Caso de uso concreto:

- Admin del server construye una autopista de 10km con .dze
- Graba una ruta con PathLogger sobre esa autopista
- El bus de BrigadaZ_Transport corre sobre la autopista usando esa ruta
- Pasajeros del server pueden viajar entre dos puntos lejanos del mapa

Por qué es importante:

1. **Sinergia entre comunidades de modding**: modders de mapa

(constructores) y modders de AI (nosotros) se necesitan mutuamente. Una autopista sin AI driving es solo decoración. AI driving sin infraestructura custom queda limitado a las carreteras vanilla.

1. **Framework como plataforma, no solo como mod**: el mod BrigadaZ_Transport

demuestra el bus costero, pero el **framework** debajo habilita cualquier sistema de transporte que un admin imagine.

1. **Distribución modular**: el framework + las pistas custom + las rutas

grabadas son tres capas distribuibles independientemente. Un modder puede tomar dos y agregar la tercera (su autopista, su ruta).

Para el paper: posicionar el framework como **enabler** de gameplay emergente. No es "un mod de bus" — es "el sistema que cualquiera puede usar para hacer transporte AI en su mundo custom". Vale como sección de "futuras aplicaciones" o cierre del manual.

Reflexión filosófica importante (*textual de Sonom4n*):

eAI no es adversario. Es sistema cooperativo cuyo comportamiento es **determinístico y conocible**. La frustración previa ("la IA es lenta", "corta las curvas") venía de tratarla como agente con voluntad propia que se resiste. La reformulación correcta: eAI es la herramienta perfecta para reproducir intenciones, **una vez que entendemos cómo traduce inputs en comportamiento**.

Para el paper: esto reescribe el roadmap. Los fixes anteriores (lookahead adaptativo, MAX_BRAKE_DECEL tuning) son válidos pero **aproximaciones a falta del patrón**. Una vez caracterizado eAI, esos fixes pueden volverse innecesarios o reemplazarse por compensación directa. **Vale como capítulo o sección destacada**: "del control adaptativo a la caracterización del sistema controlado". Es la maduración técnica más profunda del framework.

Refinamiento metodológico — batch vs iterativo (*observación de Sonom4n post-validación 2026-05-22*):

Aplicamos un approach **batch**: $n=3$ tomas $\rightarrow$ un análisis $\rightarrow$ un fix grande $\rightarrow$ 1 toma de validación. Resultado: el peor punto (zigzag wp 543) mejoró 12%, pero otras zonas (rotonda Chernobyl) quedaron sin cambio porque el mismo fix no las cubría adecuadamente.

El approach **iterativo** (1 toma $\rightarrow$ análisis $\rightarrow$ fix chico $\rightarrow$ 1 toma $\rightarrow$ análisis $\rightarrow$ fix chico $\rightarrow$...) es probablemente superior para este caso:

- En batch: justificamos $n=3$ con "distinguir señal de ruido". Pero en control iterativo, **el ruido se filtra a través del feedback continuo**, no por simultaneidad de muestras. Un volantazo aleatorio en toma N probablemente no se repite en toma $N+1$, autodescartándose.
- En iterativo: cada fix ataca el peor problema observado. La siguiente toma revela el nuevo peor problema (porque el anterior ya está resuelto). Convergencia paso a paso al óptimo.
- En batch: un único brochazo que cubre varias zonas pero ninguna perfectamente. Lo confirmamos empíricamente — zigzag mejoró, rotonda no.

Implicancia para el ILC automatizado (v2.0): el loop debería ser **iterativo, una toma a la vez**, NO batch. Cada ciclo:

1. Bus hace una corrida (PathLogger AI activo)
2. Framework compara contra recording humano
3. Identifica el wp con MAYOR desviación sistemática
4. Ajusta un solo parámetro (lookahead local en ese wp, o brake en zona X) en pequeña magnitud
5. Próxima corrida $\rightarrow$ revisión $\rightarrow$ siguiente ajuste

Esto es **el método clásico de Iterative Learning Control**, no nuestra versión simplificada de batch. Vale para el roadmap v2.0: especificar ILC iterativo (no batch) como diseño.

Reflexión para el paper: lo aprendimos haciéndolo mal primero. La metodología iterativa es más eficiente, pero requiere más iteraciones totales. Cuando es humano dirigiendo (cada iteración = build + test manual de 10 min), batch es razonable como compromiso. Cuando es el framework auto-tuneando (cada iteración = corrida automática + recálculo de parámetros, sin intervención humana), iterativo es claramente mejor.

El humano (vos) intuyó esto desde el principio. Yo justifiqué batch con argumentos estadísticos que no aplican al caso de control. La intuición operativa del usuario era más correcta que el razonamiento formal del LLM. Vale anotarlo como observación: **el experto del dominio a veces tiene razón aunque no pueda articularla en términos formales, y vale escucharlo**.

Trade-off precisión vs fidelidad de velocidad (*análisis 2026-05-22, 3 tomas IA vs grabación humana*):

Tras grabar 3 corridas de la IA con `MAX_BRAKE_DECEL=50` (v2.7) y compararlas con la grabación humana original, encontramos divergencias sistemáticas:

Métrica	Humano	IA (avg 3)	Δ
MaxSpeed en rectas	78-80 km/h	66-72 km/h	-8 a -14 km/h
AvgThrottle	0.47-0.75	0.52-0.80	+0.05-0.10
AvgBrake en crucero	0.04-0.07	0.01-0.02	-0.04

Lectura inicial errónea: "la IA es peor que el humano, hay que corregir".

Lectura correcta (corregida por Sonom4n): la divergencia es la **consecuencia inevitable del trade-off ya elegido** (precisión sobre realismo). El control predictivo agresivo (MAX_BRAKE_DECEL=50) que nos permite llegar exacto al cartel **necesariamente** reduce la velocidad de crucero y elimina las frenadas anticipadas del humano. Si subiéramos las velocidades para "igualar al humano", romperíamos el norte:

- Más velocidad en crucero → más velocidad llegando a curvas → pérdida de maniobra en zigzag y rotonda → mal llegada a paradas
- Reintroducir `targetBrake` del recording → volvemos a Approach 1 que nos dio 8 iteraciones de problemas

Consistencia entre tomas IA: las 3 tomas variaron entre sí 0.5-3 km/h. **Muy reproducible**, lo cual confirma que la IA no es errática — es consistentemente conservadora vs el humano, y esa conservación es **lo que hace que llegue exacto a las paradas**.

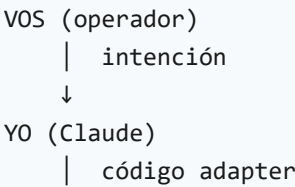
Para el paper: este es un trade-off arquitectónico explícito de cualquier sistema de playback con control adaptativo:

- **Modo "fidelidad alta":** replica al humano en velocidades, pierde precisión final
- **Modo "precisión alta":** llega exacto a destino, sacrifica realismo de manejo

No es "uno u otro mejor", es elección de diseño según caso de uso. Para nuestro caso (jugador agazapado, escapar de zombies), precisión gana. Para otros casos (turismo, simulación de manejo realista), fidelidad podría ganar. **El framework debería permitir elegir vía parámetro** (modo CONSERVATIVE vs FAITHFUL) en el JSON de configuración. Eso queda para v1.x.

Observación meta — el patrón es fractal (2026-05-22, articulado por Sonom4n):

El mismo principio arquitectónico ("no pelear con la capa inferior, tratarla como sistema cerrado con API conocida, inyectar inputs") **se aplica recursivamente en cada nivel del proyecto**, sin que lo decidiéramos conscientemente:



```

↓
FRAMEWORK (BZBusService)
|   SetThrottle/SetBrake/SetSteering
↓
eAI (sistema cerrado)
|   comandos de vehículo
↓
DayZ Engine

```

Y dentro del runtime se repite:

```

RECORDING (intención del operador)
|
↓   control predictivo (adapter)
↓
TOMA 2 calibración (caracterización del vehículo)
|
↓   parámetros del adapter
↓
eAI

```

Cada nivel:

- No modifica al inferior internamente
- Lo trata como caja negra con API conocida
- Le inyecta lo necesario para que cumpla la intención de arriba

Para el paper: vale como observación de cierre. El proyecto no solo demuestra una técnica (adapter sobre AI scripted) — la **encarna en su propio proceso de desarrollo**. La metodología es coherente con la arquitectura. Probablemente esto refleja un principio más general: las soluciones de software que escalan tienden a tener simetría entre cómo están hechas y cómo fueron hechas. "Eat your own dog food" llevado al nivel arquitectónico.

Apéndice B — Roadmap del framework

- v1.0: bus costero (este mod) — demo del patrón
- v1.1: multi-ruta vía JSON
- v1.2: adapter BoatScript
- v1.5: framework `BZAIVehicleService` genérico

Apéndice C — Créditos externos y acknowledgments

Paradas físicas del bus: @Brutalist Bus Stops

Los modelos de las paradas del bus en el server BrigadaZ son del mod **@Brutalist Bus Stops** de **Buddy** (docbuddy en Discord). En la versión actual del Transport el mod externo se carga como dependencia opcional; en la versión a publicar en Workshop los modelos van a estar repackeados dentro de BrigadaZ_Transport con crédito explícito al autor (autorización expresa otorgada por Buddy el 2026-05-14).

Importante: la UI del Transport **no depende** de los modelos físicos. El detector de proximidad usa solo coordenadas + radio definidos en el JSON. Si el mod externo no está cargado, las paradas no se ven en el mundo pero la mecánica del servicio funciona igual (F cerca de la coordenada de una parada abre la UI).

Vehículos de prueba

Durante el desarrollo del framework usamos vehículos de varios mods de la comunidad del Workshop para validar que la herencia automática de `CarScript` funciona con mods de terceros (ver Capítulo 3). Probamos autos deportivos, off-road y camiones de distintos autores — todos funcionaron sin escribir una línea adicional de código por vehículo, lo que confirmó la teoría de que el framework cubre el 95% de los car mods del Workshop.

No hicimos repack de ningún vehículo. Los mods se cargan como dependencias del server durante pruebas; el framework no contiene código de terceros.

Asistente AI

Ver [Apéndice A](#) para el disclosure completo del uso de Claude Opus 4.7 (Anthropic) como asistente técnico durante el desarrollo del framework y este manual.

Apéndice D — AI knowledge pack para modders

Este apéndice referencia un archivo separado: `MOD_CONTEXT_FOR_AI.md` en la raíz del repo. Es un documento diseñado para ser adjuntado al workspace de un asistente IA (Claude / GPT / Gemini / Opus / otro) junto con los archivos del mod, para que esa IA tenga el contexto técnico completo del framework y pueda asistir respondiendo preguntas con precisión.

Por qué este archivo existe

Una de las preguntas más frecuentes que vamos a recibir de la comunidad es "cómo aplico este patrón a mi server / mi mod / mi caso específico". Esas preguntas no son siempre genéricas — dependen del código del modder, su setup particular, qué versión de Expansion usa, qué otros mods tiene. Es contenido que se responde mejor con un asistente IA que tenga acceso simultáneo al workspace del modder Y al contexto del framework.

Nosotros desarrollamos este framework usando ese setup (VSCode + Claude Opus 4.7). Pasar la metodología a otros modders es parte del deliverable.

Cómo usarlo

Si sos modder y querés que tu asistente IA te ayude con BZ_Transport:

1. Copiá `MOD_CONTEXT_FOR_AI.md` a la raíz de tu workspace en VSCode
2. Asegurate de que el asistente IA pueda ver ese archivo (Cline / Continue / Roo / Aider / Cursor / Claude Code lo agarran si está en el repo)
3. Pedile a tu asistente: `""Leé MOD_CONTEXT_FOR_AI.md antes de responder preguntas sobre el mod""`
4. Hacé tus preguntas. El asistente va a tener contexto sobre la arquitectura, los gotchas, los archivos críticos, las constantes ajustables, y los workflows.

Si sos otro modder publicando un mod basado en este framework:

Incluí tu propia versión adaptada del `MOD_CONTEXT_FOR_AI.md` en tu repo. Es una práctica que recomendamos para mods complejos: facilita muchísimo el soporte a usuarios y reduce la carga sobre vos.

Formato del archivo

Es markdown denso, organizado por:

- Información esencial (identidad, stack, dependencias)
- Estructura del repo y archivos críticos
- Arquitectura del framework (diagramas de flujo, singletons, RPCs)
- Pipeline de grabación de rutas
- Configuración por JSON
- Constantes ajustables con sus valores y significados
- Compatibilidad de vehículos (qué funciona out-of-the-box)
- Gotchas conocidos en orden de probabilidad
- Cómo ejecutar el mod paso a paso
- FAQ con respuestas que el asistente puede dar directamente
- Roadmap de versiones
- Limitaciones del soporte IA (qué casos derivar al humano)

Limitaciones

Este archivo no reemplaza al manual humano (este `PAPER_NOTES.md` → futuro `MANUAL_eAI_VEHICLES.md`). Está optimizado para consumo de IA, con menos narrativa y más estructura. Un humano leyendo este archivo va a encontrarlo seco y denso. Para entendimiento conceptual del framework, leer el manual humano.

Apéndice E — Sobre los autores y el origen del proyecto

Los autores

Este framework y este manual fueron desarrollados por **Sonom4n** e **Hiperhipo10**, dos amigos sin formación previa en programación profesional, durante el primer semestre de 2026.

Nuestra contribución a la comunidad de modding de DayZ surge de un proyecto personal: armar un server propio con contenido custom para nuestros amigos. No éramos modders profesionales antes de empezar, y no nos consideramos tales ahora. Lo que terminamos creando — un framework que destraba una capacidad histórica de DayZ — es el resultado de iteración persistente, asistencia de IA, y una pizca de suerte al elegir el ángulo correcto del problema.

El origen del proyecto

El 6 de febrero de 2026, Hiperhipo10 propuso armar un server DayZ Standalone. Arrancamos sin un plan grande: solo queríamos un server PVE para jugar entre amigos, con algo de contenido custom para que se sintiera distinto.

Durante los primeros meses fuimos desarrollando una serie de mods que cubrían distintas necesidades del server, aprendiendo el pipeline de modding de DayZ en el proceso (Enforce script, mod Expansion, eAI, AddonBuilder, deployment):

- **@BrigadaZRadio** — sistema de radio para vehículos y radios de mano con 6 estaciones, playlists configurables vía JSON, contenido propio: publicidades de los traders del server, anuncios de zonas y **anticipos de programas radiales ficticios** con tono humorístico (programa de caza, de turismo en Chernarus, de mascotas con "consejos para domesticar animales salvajes", de construcción, etc. — los programas no existen, son sketches de pocos segundos que agregan personalidad al server)
- **@BrigadaZ_Info** — UI informativa contextual ubicada en la puerta del pescador y del cazador con info de equipamiento, peces, zonas de pesca y caza para jugadores que aún no exploraron esas mecánicas
- **@BrigadaZ_Finanzas** (en desarrollo) — sistema financiero con recotización de activos (RUB / EUR / USD / XAU / BTC) en cada reinicio del server, y trader de activos que rota entre ubicaciones para evitar campeo previo al reinicio
- **@Losrollinmod v1** — retexturado de carteles del mapa con gráfica y publicidad propia del server
- **@Losrollinmod-v2** — pantalla de presentación del server usando Expansion
- **@BrigadaZ_Mission** (en desarrollo) — extensión del sistema de Quest de Expansion para misiones más allá de las limitaciones del módulo, intentando emular las antiguas misiones de Epoch/Overpoch (claim de AI camp, heli crashes con loot box, marca con flare, etc.)

En algún punto de ese proceso, conversando sobre qué le faltaba al server para sentirse "vivo", se nos ocurrió la idea de un servicio de bus público que recorriera la costa de Chernarus. Pensábamos que iba a ser un mod más en la lista. No fue.

Por qué este proyecto terminó siendo distinto

Cuando nos pusimos a implementar el bus, descubrimos que el problema no era de gameplay sino de capacidad técnica: en DayZ Standalone, los NPCs no manejan vehículos de forma utilizable. Pasamos las semanas siguientes intentando los workarounds que estaban documentados en foros, copiando ideas de mods abandonados, peleando con el motor.

Tras varios callejones sin salida, dimos con un breakthrough técnico (documentado en el Capítulo 2) que destrabó el problema. Al darnos cuenta de que la solución era reutilizable más allá del bus específico que queríamos hacer, decidimos transformar el proyecto en algo más grande: un framework reusable + manual público + AI knowledge pack para otros modders. El bus de la costa quedó como demostración funcional del patrón.

Metodología de trabajo

Usamos un setup de desarrollo asistido por IA: **VSCode + Claude Opus 4.7 (Anthropic)** como pair-programmer. La división de roles fue:

- **Nosotros** : visión del producto, diseño de gameplay, conducción de pruebas in-game, diagnóstico empírico, decisiones estratégicas, dirección sesión a sesión.
- **El asistente IA**: implementación de código en Enforce, exploración de fuente de terceros (notably eAI y Expansion), drafting de este manual, síntesis técnica.

El breakthrough del Capítulo 2 emergió de esa colaboración: la hipótesis "eAI mismo puede estar hardcodeando el gear" fue nuestra; la verificación en el código de Expansion y la implementación del override fueron del asistente.

Recomendamos este setup a otros modders. El Apéndice D incluye un AI knowledge pack diseñado para que cualquiera pueda reproducir este tipo de colaboración con su asistente preferido (Opus, GPT, Gemini, otros) sobre nuestro framework.

Disponibilidad y soporte

El mod se publica en Steam Workshop como **BZ_Transport (Bus eAI Driving)**. El código y el manual están disponibles en GitHub bajo licencia abierta. Para preguntas, modders pueden:

1. Adjuntar el AI knowledge pack (Apéndice D) a su workspace y preguntarle directo a su asistente IA
2. Abrir issues en el repo de GitHub
3. Contactarnos en Discord para casos puntuales

Esperamos que este trabajo sea el principio de una nueva categoría de contenido en DayZ Standalone, no su fin.

Licencia, uso libre, donaciones

No comercializamos este proyecto. El mod, el framework, el manual y todos los archivos que lo acompañan son completamente libres para uso, repack, rebuild, fork, modificación y redistribución. No pedimos permiso ni atribución obligatoria (aunque siempre se agradece). El único pedido es que los componentes de terceros incorporados (como los modelos de @Brutalist Bus Stops de Buddy/docbuddy) mantengan sus propias condiciones de uso — si querés repackearlos, contactá con el autor original.

Invitamos a la comunidad de modding a mejorar, reconstruir, desarrollar y potenciar este framework. Mejor que un mod particular es un patrón que muchos adoptan, mejoran y reinterpretan para casos que nosotros nunca imaginamos.

Si en algún momento querés aportar para que podamos seguir trabajando en este tipo de proyectos, dejamos un link de donaciones voluntarias (PayPal):

<https://paypal.me/Sonom4n>

Cualquier aporte va a tiempo de desarrollo de futuras versiones (v1.1, v1.2, adapters de boat / heli, framework genérico). Sin presión — el mod es libre con o sin donaciones.

Notas y placeholders pendientes

(Marcas para insertar assets cuando estén disponibles. Borrar cuando se redacte la versión final.)

- **Video 1:** primera prueba de bot driving cuando aun no aceleraba —
<https://youtu.be/XhoboTTzaTM> → demuestra el síntoma original del blocker, contexto para Capítulo 2
- **Video 2:** bot manejando por la ruta con PRIMERA clavada —
<https://youtu.be/FsWfmXmDHxg> → mismo bloqueante en una grabación posterior
- **Video 3** *(pendiente de URL — grabado por Sonom4n, 2026-05-21):*
bus llega cerca del cartel de Kamenka pero se queda **~55m antes** del waypoint stop y no avanza. Causa: playback literal del recording reproduce el frenado anticipado del operador humano; bus sin pasajeros frena más rápido que el humano y queda paralizado lejos del stop, sin disparar OnWaypointReached(isStop=true). Evidencia visual del problema que motivó la arquitectura híbrida (recording = forma+velocidad, controlador procedural = pedales en zona crítica). → Capítulo 4.4 / Capítulo 5.5 — caso de estudio "del playback literal al control híbrido". Pareja con Video 4 (post-fix) cuando se grabe.
- **Video 4** *(a grabar después del fix):* misma vuelta con el modo parking activo (rampa de freno lineal entre 40m y 3m). Comparación side-by-side con Video 3 cierra el caso de estudio.

Timeline de iteración del control de freno (sesión 2026-05-21)

(Esta sesión completa fue grabada por Sonom4n. Cada hora de build corresponde a un cambio de comportamiento visible. Mapeo para cruzar con los videos: buscar timestamp en filename o duración relativa.)

Hora build	Versión	Qué cambió	Comportamiento esperado en video
20:38:51	v0	Playback literal (targetBrake del recording)	Bus frena por anticipación humana, se queda ~55m antes del cartel, nunca llega (= Video 3 original)
21:37:02	v1.5	Modo crucero + parking rampa lineal (40m → 3m)	Bus llega más cerca pero se queda ~20m del cartel ("stop fake"), espera 30s, después crawl al cartel
21:51:38	v1.6	Detección m_AtStop por distancia (no por wp isStop)	Mismo visual que v1.5, ya no se atasca en loop muerto. Cambio arquitectónico, casi imperceptible en video
22:18:49	v2	Control físico predictivo MAX_BRAKE_DECEL=6.0	Frenado más natural (suave inicio, fuerte en peak), llega a 3-4m del cartel pero le falta empujoncito
22:40:07	v2.1	Empuje 0.2 → 0.35 + stopDuration 30 → 7	Bus llega al cartel completamente, dwell de 7s. Peak de freno todavía a ~20m del cartel
22:59:20	v2.2	MAX_BRAKE_DECEL 6.0 → 9.0	Peak de freno ~10m del cartel (más natural visualmente)
23:15:03	v2.3	MAX_BRAKE_DECEL 9 → 12	Peak ~8m del cartel, llegada a 2m, frenado más concentrado
23:34:03	v2.4	MAX_BRAKE_DECEL 12 → 16	Peak ~7m, llegada a 1.5m, freno "más duro"
23:52:47	v2.5 ★	MAX_BRAKE_DECEL 16 → 20	VALOR DE REFERENCIA: llega 1m antes del cartel, aceptable para el caso de uso (precisión sobre realismo). Good known state validado 2026-05-22.
2026-05-22 19:28	v2.6	MAX_BRAKE_DECEL 20 → 30	Bus no se pasa del cartel, se queda corto (hipótesis: drag domina, o bus frena más fuerte de lo asumido).
20:20:58	v2.7	MAX_BRAKE_DECEL 30 → 50	Frenado perfecto en paradas. Pérdida notable de maniobra en curvas. Reframing del norte (precisión > realismo) → este es el valor adoptado.
21:40:53	v1.1	AI logging server-side (NUMPAD 7 toggle, CSV por tick)	Permite grabar trayectoria del bus AI para comparar con recording humano. Fix de filename con sufijo _t<tickMs> para unicidad entre tomas.

Hora build	Versión	Qué cambió	Comportamiento esperado en video
23:13:42	(3 tomas IA grabadas)	Análisis lateral identifica 8 wp problemáticos	Zigzag Kamenka→Komarovo wp 543 (avg 3.93m max 4.79m) + wp 605 (2.22m). Rotonda Chernov wp 1557-1577 (~2.2m). El control con <code>LOOKAHEAD_DIST=10</code> fijo "cortaba" las curvas.
23:41:08	v1.2	Lookahead adaptativo en <code>DriveTowards</code>	<code>ComputeLocalCurvature()</code> mide cambios de heading en próximos 10 wps. Lookahead interpola entre 10m (recta) y 5m (curva fuerte). No toca velocidad ni precisión de stops — solo mejora steering en curvas. Pendiente: validar con toma 4.

Reinicios del server durante la sesión: el server se reinició después del build 22:42 (cambio de `stopDuration` 30 → 7 que requería relectura del JSON). Antes, los stops eran de 30s en todos los videos.

Para el paper: esta progresión es el caso de estudio "del playback literal al control predictivo". 6 iteraciones, ~2 horas, cada una empíricamente justificada. Material narrativo fuerte para Capítulo 4.4 o Apéndice nuevo "Evolución del control".

Robustez emergente (*observación durante prueba 2026-05-21 ~23:20*): en una de las vueltas el bus encontró dos buses de sesiones anteriores sobre la ruta en Chernov. El bus actual los chocó, se quedó detenido contra ellos (kmh $\approx$ 0, ~30m del cartel siguiente). Cuando se borraron los obstáculos, el bus **retomó la marcha solo** y llegó a la parada sin intervención. No programamos recovery explícito — el sistema lo manejó porque el control declarativo no asume que el bus se mueve siempre: si está casi parado dentro de la zona parking lejos del cartel, el branch de empuje (`throttle = 0.35`) ya está activo. Cuando el obstáculo se liberó, el throttle existente hizo el resto. **Argumento defensivo para el approach declarativo vs FSM rígida:** cada estado del vehículo (parado, crucero, frenando, empujando) tiene un branch que lo trata, sin asunciones implícitas sobre el flujo. Vale para el capítulo de arquitectura o el cierre del de control.

Tema lateral: los buses fantasma de sesiones anteriores son un problema separado — al startup no se hace cleanup de instancias previas de `ExpansionBus` huérfanas en el mundo persistente. Solución eventual: en `BZBusService.Init()` buscar y borrar todos los `ExpansionBus` antes de spawnear el nuevo. Pendiente para v1.x si molesta en producción.

Reframing del objetivo de frenado (*decisión de diseño 2026-05-21 ~23:50*): durante la sesión iteramos buscando que el frenado se **sintiera humano** (suave, modulado, pulsado). Pero el contexto del juego invalida esa métrica: el jugador en la parada está **agazapado escapando de zombies**. Lo que necesita del bus es **precisión de llegada y rapidez de paso**, no realismo de manejo. Cuanto más tiempo el bus tarde en frenar suave y elegante, más tiempo el jugador está expuesto.

North star revisado: "el bus se detiene **exactamente** en el punto marcado del recording, con la mínima latencia posible". Frenado fuerte y abrupto es preferible a frenado suave y largo. Esto invalida los approaches de "humanización" (cap suave, freno pulsado) que estábamos considerando — son optimizaciones para el problema equivocado.

Consecuencia técnica: subir `MAX_BRAKE_DECEL` agresivamente hasta encontrar el sweet spot donde el bus llega exacto sin pasarse. En el límite, el approach converge a "bang-bang control" (freno fondo

desde el punto óptimo, cero antes) — clásico de control teórico cuando priorizás precisión sobre suavidad. Vale como nota técnica en el capítulo de control: "la decisión entre control suave vs bang-bang depende del contexto del usuario final, no de la elegancia matemática".

Para el paper: sirve como ejemplo de cómo el contexto del gameplay puede redirigir decisiones técnicas que parecían cerradas. Una hora de tuning para humanizar fue trabajo bien iterado pero en la dirección equivocada — sin la observación del usuario sobre el caso de uso real (zombies + paradas), habríamos llegado a un bus elegante que dejaba morir gente.

Invariancia al peso del pasajero (*observación 2026-05-21 ~23:35*): probamos la misma vuelta vacía y con un jugador a bordo. Comportamiento **idéntico** — el bus para en el mismo punto del cartel, con la misma curva de frenado, en ambos casos. Esto valida una propiedad importante del control físico predictivo: como la fórmula $\text{brake} = u^2 / (2 \cdot s \cdot \text{MAX})$ es agnóstica a la masa del vehículo y depende solo de velocidad observada y distancia restante, la masa adicional del pasajero se "resuelve sola" — el cálculo en el siguiente tick ve velocidad ligeramente distinta y ajusta el brake automáticamente. **Esto NO pasaría con el playback literal** (Approach 1) donde el `targetBrake` del recording fue grabado con una masa específica. Argumento técnico fuerte: el control predictivo es **escalable a vehículos con carga variable** sin recalibración. Aplica a buses, camiones, vehículos militares con tropas, autos con remolques. Vale como párrafo en Capítulo 4.4 o como ventaja enumerada en la sección de approaches.

Principio fundamental — fidelidad al recording sobre realismo impuesto ^{*}(refinamiento de diseño 2026-05-22,

El framework **no impone un estilo de manejo**. Reproduce el del operador.

- Si el operador grabó frenada fuerte y corta (por razones de gameplay — escapar de zombies, llegar rápido a destino), el playback reproduce frenada fuerte y corta.
- Si grabó frenada suave (turismo, realismo de simulación), reproduce suave.
- Si grabó pasando por la banquina en una curva, reproduce pasando por la banquina.

Implicancia para el control predictivo: el approach 3 (control físico predictivo) que iteramos hoy NO es la arquitectura final correcta. Es una **capa de compensación** para cuando el recording es insuficiente. Típicamente esto pasa con **input device discreto** (teclado: throttle y brake son 0 o 1, no hay valores intermedios). Con joystick analógico o pedales reales, el recording captura modulación humana y el playback literal del recording debería alcanzar.

Re-jerarquía correcta de approaches:

1. **Approach 1 — Playback literal:** principal cuando el recording es rico (joystick, pedales analógicos). Reproduce fielmente.
2. **Approach 3 — Control predictivo:** safety net cuando el recording es pobre (teclado) o cuando hay divergencia física (peso muy distinto, etc.).
3. **Approach 2 — Rampa lineal:** descartado, lo dejamos solo como ejemplo didáctico de "el camino del medio".

El rol de la toma 2 (calibración): no es para *decidir cómo manejar*, es para *describir el vehículo*. La toma 1 (recording) describe **qué quiere el jugador**; la toma 2 (calibración) describe **cómo responde el vehículo**. La combinación permite playback fiel sin necesitar intervención del control predictivo.

Re-framing del proyecto (*versión final*):

- Framework de **reproducción de intención** para entidades guiadas por AI
- El operador define la **intención estratégica** durante la grabación (qué ruta seguir, cómo querer que se frene, cómo querer pasar las curvas)
- El framework adapta la **táctica** para realizar esa intención en las condiciones del momento (peso real, lluvia, asfalto, daño del vehículo, obstáculos imprevistos)
- El control predictivo, modo parking y demás controladores procedurales son **herramientas tácticas** al servicio de la intención grabada, no decisiones autónomas del framework

Generalización potencial — AI a pie y otras entidades (2026-05-22): si el PathLogger graba posiciones + inputs de cualquier entidad controlada por eAI (no solo vehículos), el framework se generaliza a:

- Patrullas a pie con rutas tácticas grabadas
- Líder de squad que graba táctica (cubrirse acá, flanquear allá)
- Cualquier movimiento que hoy se hace con waypoints rígidos

El principio "intención vs táctica" aplica idéntico: el operador define qué quiere que pase, el framework lo realiza dadas las condiciones. Extiende el scope del paper de "vehículos guiados" a "entidades guiadas".

Factores ambientales a considerar en táctica (*roadmap*):

- Peso real del vehículo (carga + pasajeros) → ajusta deceleración esperada
- Lluvia / asfalto mojado → reduce coeficiente de fricción → frenar más temprano
- Viento fuerte → afecta resistencia aerodinámica → ajusta throttle
- Daño del vehículo (motor, ruedas) → reduce capacidades → recalibra MAX_BRAKE/MAX_ACCEL
- Obstáculos dinámicos (otros vehículos, AI, players) → modifica trayectoria
- **Terreno e inclinación** (*observación 2026-05-22*): la pendiente del

terreno afecta tanto la necesidad de freno como de aceleración. En una bajada, soltar acelerador NO es suficiente — la gravedad acelera el vehículo y se necesita frenado preventivo. En una subida, frenar agresivo es innecesario — la gravedad ayuda a desacelerar. En curvas el vehículo pierde adherencia y necesita más freno para mantener trayectoria. **Cómo medirlo en runtime**: leer la altura Y del bus y del próximo waypoint (`pos[1]`), calcular pendiente local, ajustar el `MAX_BRAKE_DECEL` efectivo en función del grade (subida +20%, bajada -20%). Esto está disponible "gratis" del waypoint data sin configuración adicional.

Cada uno de estos es una dimensión donde el framework tiene que adaptar la táctica para preservar la intención grabada.

Para el paper: este es **el** principio rector del framework, vale como tesis. Toda la arquitectura se deriva de esta decisión. Lo separa de cualquier sistema de AI driving que impone su propio modelo de manejo

(eAI vanilla, ARMA AI, etc.) — esos sistemas **deciden** cómo manejar; este sistema **reproduce** cómo manejó alguien.

Auto-tuning loop — el framework se ajusta solo *(visión arquitectónica 2026-05-22,

Lo que estamos haciendo hoy es **feedback loop manual**: el operador graba un patrón padre, la IA lo reproduce, el operador compara y ajusta parámetros, build nuevo, repetir. Las 8 iteraciones de la sesión (v0 a v2.7) son exactamente eso, con el desarrollador como loop de feedback.

La visión a futuro: que ese loop sea **automático**. El framework:

- 1. Toma el recording humano como referencia (patrón padre)
- 2. Hace una corrida de playback grabándose a sí misma (AI logging)
- 3. Compara la trayectoria observada vs el recording referencia
- 4. Calcula error (delta posicional, delta velocidad por tramo)
- 5. Ajusta parámetros internos (MAX_BRAKE_DECEL, throttle de empuje, PARKING_THRESHOLD) para minimizar el error
- 6. Próxima corrida → mejor aproximación → repetir hasta $\Delta < \text{threshold}$

En literatura de control esto se llama **Iterative Learning Control (ILC)** o **Model-Reference Adaptive Control**. Diseñado justo para tareas repetitivas donde cada iteración informa la siguiente.

Cómo se unifica con el resto de las visiones:

Componente	Rol
Toma 1 (calibración)	Caracteriza el vehículo (parámetros físicos)
Toma 2 (ruta)	Define la INTENCIÓN del operador
AI logging	Mide la EJECUCIÓN real de la IA
Auto-tuning loop	Cierra el ciclo: minimiza delta intención-ejecución

Workflow del usuario final del framework v2.0 (ideal):

- 1. Spawneá tu vehículo, hacé la calibración (2 min)
- 2. Hacé tu grabación de ruta (variable)
- 3. Activá "auto-tune" → el framework hace 3-5 corridas internas, comparándose con tu grabación, ajustando solo
- 4. Listo. El JSON de configuración del vehículo queda completo, sin intervención manual.

Para el paper: esta es la maduración técnica del framework. Pasamos de "herramienta de modder con muchos parámetros para tunear" a "sistema auto-adaptable que aprende a reproducir intenciones". Vale como cierre del paper o como roadmap v2.

Camino de implementación (alto nivel):

- v1.0 actual: tuning manual, parámetros hardcoded

- v1.1: PathLogger AI logging implementado (comparación manual)
- v1.2: Tomar 2 (calibración) implementada (parámetros físicos por vehículo)
- v2.0: ILC loop completo (ajuste automático de parámetros adaptativos)

Cada paso construye sobre el anterior. Hoy estamos cerrando v1.0 y arrancando con v1.1 (AI logging para análisis manual).

PathLogger como herramienta de provisioning del framework (*visión arquitectónica 2026-05-22, propuesta de Sonom4n*):

Hasta ahora el PathLogger es "una herramienta auxiliar para grabar rutas". La visión correcta es elevarlo a **componente fundamental del framework**: herramienta de provisioning automática para cualquier vehículo.

Workflow propuesto del usuario final del framework:

1. Toma 1 — Calibración del vehículo (~2 min):

- Frenada full desde velocidad alta → mide `MAX_BRAKE_DECEL` real
- Aceleración full desde 0 → curva de aceleración + `a_max`
- Coast test (sin throttle) → drag/inercia natural
- Shift points (NUMPAD 3/0, ya implementado) → gear ranges
- Tiempo de respuesta del input (step input) → delay del control
- Output: `vehicle_calibration.json` con todos los parámetros físicos medidos empíricamente del vehículo específico

1. Toma 2 — Grabación de ruta (variable según ruta):

- Lo que ya hace el PathLogger actualmente
- Output: `route_<name>.json` con waypoints + inputs

1. Listo: el framework lee ambos JSON y opera sin más configuración.

Impacto en el control predictivo:

- Las 6 iteraciones que hicimos para encontrar `MAX_BRAKE_DECEL = 20` desaparecen — el bus mide su propia deceleración en la toma 1 y la usa.
- El control es **universalmente válido** para cualquier vehículo, no solo el ExpansionBus.
- Eliminamos el riesgo de "vehículo nuevo = recalibrar todo a mano".

Cambio de framing del proyecto:

- ANTES: "framework para buses con AI driving"
- DESPUÉS: "sistema universal de vehículos guiados auto-calibrante"

Esto convierte el manual del usuario en una página:

Implementación pendiente (no urgente, roadmap v1.x):

- Diseñar el formato de `vehicle_calibration.json` (extendiendo

`gear_ranges.json` con los nuevos parámetros)

- Implementar los modos de calibración con hotkeys (NUMPAD 1/2/4/5/6/7):
- 1: Start acceleration test
- 2: Start brake test
- 4: Start coast test
- 5: Start input step test
- 7: Save calibration JSON
- HUD simple que indique al operador qué test está corriendo y los resultados (deceleración medida, etc.)
- Modificar el control predictivo en `DriveTowards` para leer el `MAX_BRAKE_DECEL` del JSON por `vehicleClass`, fallback a 20.0 si no hay calibración

Para el paper: esta es la visión final del framework. Vale como capítulo final o apéndice de roadmap. Probablemente es lo que diferencia el mod de cualquier otra cosa parecida — el resto del trabajo arquitectónico de control predictivo + recording + playback solo adquiere su valor pleno cuando se combina con auto-calibración.

Recording como CORREDOR, no como línea:

Se nos ocurrió la siguiente observación filosófica: el bus, como un vagón sobre rieles, no sigue una línea central, sigue dos referencias paralelas. Esa diferencia conceptual es importante.

Es completamente válido técnicamente. Artículo:

Approach actual (línea 1D):

- PathLogger graba un punto + heading por sample
- Recording = secuencia de puntos = **línea**
- Playback intenta llegar exacto a cada punto
- Desvío lateral = "fuera de la línea"

Approach propuesto (corredor 2D):

- Mismo recording (no requiere modificación), pero **interpretado** como **par de líneas paralelas** derivadas de centro + heading + ancho del vehículo
- Playback evalúa: "¿estoy dentro del corredor?" en vez de "¿estoy en la línea?"
- Tolerancia lateral natural del ancho del vehículo (sin perder fidelidad)

Analogía del riel: un vagón ferroviario no sigue una línea central, sigue dos rieles paralelos. Eso da estabilidad lateral garantizada. Una línea central permitiría descarrilar lateralmente "sin salirse técnicamente" de la trayectoria.

Implementación técnica: el recording actual YA tiene heading. Con heading + ancho del vehículo (~2.5m para bus), las posiciones de las ruedas se derivan matemáticamente:

```
rueda_izquierda = centro + perpendicular(heading) × (ancho / 2)
rueda_derecha   = centro - perpendicular(heading) × (ancho / 2)
```

Lo que cambia es **interpretación**, no captura.

Cambios concretos para futuro:

1. **En DriveTowards** — pure pursuit con corredor: apunta al centro del corredor lookahead, pero el `aNeeded` y el steering toleran desvío hasta ancho/2 sin corregir.
1. **En métrica lateralDev** : en vez de "distancia al wp central", medir "distancia al borde MÁS CERCANO del corredor". 0 = dentro, >0 = afuera. Mucho más útil para identificar zonas problemáticas reales.
1. **Visualización**: dibujar dos líneas paralelas en lugar de una en debug del editor. Más intuitivo.

Para el paper — esto es **shift conceptual**:

Articula el paradigma como **recording de volumen** vs **recording de trayecto**. Tiene paralelos en:

- Robótica: lane following (siguiendo carril) vs path following (siguiendo línea)
- Manejo humano real: ningún humano sigue una línea exacta — sigue el carril, hay tolerancia natural
- Navegación marítima: rumbos con tolerancia, no líneas absolutas

Es probablemente el **shift más fundamental del paradigma de playback** que hayamos articulado en esta sesión. Vale como sección arquitectónica del paper, no solo como detalle de implementación.

Roadmap implícito: implementación del "corredor" como v1.x o v2.0. La regrabación humana sigue funcionando igual (el recording no cambia), solo cambia cómo lo interpretamos.

Regrabación humana valida la hipótesis **recording-driven** (*experimento 2026-05-23/24, Sonom4n*):

Después de validar el modelo físico (sección anterior), pasamos a probar si **regrabar la trayectoria humana** mejora la fidelidad del playback, versus **tunear el control**.

Setup: el operador regrabó la ruta entera (Spawn → Chernobyl) con PathLogger humano (`path_140523-233914.csv`). Tenía 1766 muestras, 5 paradas marcadas con NUMPAD 4 (todas dentro de 1-2m de las coordenadas canónicas del cartel). Converter `csv_to_route.ps1` transformó CSV → `BZBusRoute.json` reemplazando la grabación anterior.

Resultados ingame:

Zona	Versión anterior (TOMA01-04)	Versión nueva (regrabación)
Zigzag (wp 543)	Roce sistemático ~3.93m desv	Resuelto — pasa limpio

Zona	Versión anterior (TOMA01-04)	Versión nueva (regrabación)
Rotonda Cherno	Roces ocasionales ~2.2m	Pasa OK
Curvas pronunciadas con velocidad alta	Roces de banquina	Sigue rozando
Paradas (precisión)	~1m antes del cartel	Igual ~1m antes del wp grabado

Conclusión: la **regrabación con trayectoria más amplia resuelve el zigzag** sin tocar código. El operador pasa más alejado del extremo; la IA recorta 1m por el sesgo del control predictivo, pero igual queda dentro de tolerancia. **Esto es el principio fundamental del framework validado empíricamente:** la calidad del playback escala directamente con la calidad del recording.

Lo que NO resuelve la regrabación:

- 1. **Curvas a velocidad alta:** si en la grabación el humano pasa muy rápido por una curva pronunciada (no la del zigzag), la IA reproduce esa velocidad y se sale del trazo. Necesita el approach del **corredor / rieles** para tolerar lateralmente.
- 1. **Precisión de paradas (sub-metro):** el control predictivo tiene un sesgo de ~1m menos del wp objetivo. Si el operador para "alineado con el acceso" (posición específica para subir/bajar pasajeros), la IA queda 1m antes. Necesita **ajuste fino al stop**.

Tres mejoras técnicas implementadas en respuesta:

- 1. **Bug fix de lateral_dev_m** (2026-05-24): la métrica usaba `m_WaypointIndex` directo, que es el wp objetivo del lookahead (siempre adelantado ~15m). Resultado: mean 16m de desvío que no tenía sentido físico. Corregido a búsqueda local en ventana [-30, +5] del wp más cercano. Ahora mide desvío real al trazo.
- 1. **Ajuste fino al stop** (2026-05-24): cuando `m_AtStop=true` pero distancia al wp exacto del recording es 0.5-3m, aplicar throttle suave (0.25) hasta llegar a <0.5m. Resuelve fidelidad sub-metro para usos futuros (acomodar el bus en parada exacta, garage, etc.). ~15 líneas de código.
- 1. **Corredor (rieles) pendiente:** aún no implementado. Es la solución estructural para curvas pronunciadas con velocidad. Pendiente de implementar en próxima sesión.

Para el paper — síntesis del trade-off:

La sesión validó empíricamente que la **calidad del recording** y la **calidad del control** son dimensiones ORTOGONALES:

Eje	Resuelve qué	Cómo se mejora
Recording quality	Trayectoria, intención, velocidad de crucero	Operador regrababa con mejor trazado

Eje	Resuelve qué	Cómo se mejora
Control quality	Tolerancia, precisión sub-metro, respuesta a física	Código (lookahead, factor altura, ajuste fino, corredor)

Ningún eje solo resuelve todos los problemas. **Un framework completo necesita ambos:** el operador aporta intención, el control aporta ejecución precisa.

Joystick vs teclado — la calidad del recording depende del operador, no solo del device
(observación empírica 2026-05-23, Sonom4n):

Habíamos teorizado anteriormente que **joystick analógico produciría mejor recording que teclado discreto** porque captura modulación continua de los pedales. La hipótesis era que el playback literal (Approach 1) podría funcionar mejor con recording de joystick, reduciendo la necesidad de control predictivo.

Test empírico real: el operador probó manejo con joystick y reportó:

Conclusión: la calidad del recording depende de **dos variables**, no de una:

- 1. **Riqueza del input device** (analógico > digital — teoría que manteníamos)
- 2. **Control del operador en ese device** (factor que omitíamos)

Si el operador acostumbrado a teclado no tiene práctica con joystick, **el recording de teclado puede ser superior al de joystick**, aunque el device sea técnicamente menos rico. La hipótesis "joystick = mejor" era **necesaria pero no suficiente**.

Para el paper: anotar este punto contraintuitivo. Si en el manual recomendamos joystick "para mejor fidelidad", hay que aclarar que **asume operador con práctica en ese device**. La mejor recomendación es "usá el device en el que tengas más control fino, sea cual sea".

Tema de futuro: testear con operadores que Sí tengan práctica en joystick (gamers de sims de manejo, por ejemplo). Si en ese caso joystick supera teclado, entonces la riqueza del device sí domina. Sería test de control para el paper.

Experimento pendiente — PathLogger con joystick (descartado por preferencia del operador) : hasta ahora todas las grabaciones se hicieron con **teclado** (input discreto: throttle 0 o 1, brake 0 o 1, sin valores intermedios). El PathLogger captura `targetThrottle` y `targetBrake` como `float`, pero con teclado solo recibe 0.0 o 1.0. Con un **joystick analógico** los valores serían continuos (0.3 throttle, 0.6 brake, etc.), reflejando la modulación real que un humano hace con los pedales.

Hipótesis: grabar con joystick produce un recording mucho más rico que el playback puede aprovechar directamente. Tal vez incluso elimine la necesidad del control predictivo (Approach 3) en zona parking, porque el frenado del operador ya viene modulado.

Cómo probarlo: grabar la misma vuelta con joystick, comparar el CSV de `targetThrottle` / `targetBrake` con la grabación de teclado. Si los valores son continuos y graduales

en el joystick, hacer una vuelta de playback con esos datos y ver si el bus llega al cartel sin necesidad del modo parking del control predictivo.

Implicancia para el paper: validaría una hipótesis interesante — "la calidad del playback depende de la riqueza del input device usado en la grabación". Sirve para una sección de "recommendations" en el manual ("usá joystick si querés recordings de mayor fidelidad").

Paradoja a destacar (*observación de Sonom4n durante la sesión*): **frenar nos costó más que arrancar y acelerar**. Acelerar es pisar el pedal y dejar que el motor + AT hagan su trabajo — una decisión binaria modulada. Frenar requiere anticipación, predicción cinemática de distancia/velocidad/peso, modulación fina del input, llegar a un punto preciso con velocidad cero. La asimetría no es obvia hasta que te tropezás con ella: en un sistema de seguimiento de waypoints, el control de aceleración es trivial, el control de freno es un problema de control óptimo. Vale como reflexión filosófica al cierre del capítulo de control de pedales, o como apertura del próximo. Cualquier modder que intente algo parecido va a redescubrir esta asimetría.

Hipótesis — rear-axle off-tracking en curvas pronunciadas (*observación 2026-05-24, propuesta de Sonom4n*):

Observación del operador: "Boris se piensa que es un coche, por eso las mínimas pequeñas desviaciones, notorias sobre todo en las ruedas de atrás."

Fenómeno físico: en vehículos con wheelbase largo (distancia entre eje delantero y trasero), las ruedas traseras NO siguen la misma curva que las delanteras durante un giro. Siempre "cortan camino" hacia el centro de curvatura. Cuanto más largo el wheelbase, más pronunciado el desplazamiento lateral. Se llama **rear-axle off-tracking** y es la razón por la que un colectivo abre el giro hacia afuera antes de doblar — compensación intuitiva.

Wheelbase aproximado por vehículo en DayZ:

- Hatchback: ~2.5m → off-tracking despreciable en curvas urbanas
- ExpansionBus: ~5-6m (estimado) → off-tracking ~1-2m en curvas pronunciadas (rotonda, U-turns)

Conexión con la caracterización de eAI: del Capítulo 2 sabemos que eAI vehicular trata a TODOS los vehículos con la misma fórmula de steering ($\text{turnCoef} \times \text{speedCoef} \times \text{distanceCoef} \times \text{rpmCoef}$) — no parametriza por wheelbase, longitud ni masa. Es uno de los atajos del framework eAI. Aplicar la misma lógica de steering a un Hatchback y a un bus largo necesariamente sesga al bus hacia adentro de las curvas.

Predicción falsable: si esta hipótesis es correcta, el CSV de AI logging debería mostrar **sesgo sistemático hacia el centro de curvatura** en zonas de curva pronunciada (rotonda Chernowp 1557-1577, curvas cerradas Sonomir), pero **no en rectas** ni en curvas suaves. Operacionalmente:

1. Filtrar AI logging CSV por `local_curvature > umbral` (cambio de heading acumulado en próximos N waypoints).
2. En esos tramos, verificar que el signo del `lateral_dev_m` apunta

consistentemente hacia el centro de curvatura, no es aleatorio.

3. Comparar magnitud del sesgo entre Hatchback (test) y ExpansionBus —

si la hipótesis es correcta, el bus muestra ~2-3x más sesgo.

Implicancia para el framework — compensación vía corredor: el approach del corredor (rieles, ya en roadmap) puede absorber el off-tracking de forma natural si su ancho se calcula consciente del wheelbase:

$$\text{ancho_corredor} = \text{ancho_vehículo} + (\text{wheelbase} \times \sin(\text{angulo_giro_local}))$$

Más adelante todavía: ofsetear el lookahead point hacia afuera de la curva por una distancia proporcional al wheelbase × curvatura local (equivalente al "abrir el giro" del colectivo humano). Esto es **procedural geometry** que no necesita pelear con eAI — solo desplazar el waypoint efectivo que se le pasa al pure pursuit.

Limitación de la grabación humana: cuando el operador grabó la ruta, el PathLogger capturó la posición del **morro del bus** (o del centroide, según implementación), NO la de las ruedas traseras. Por eso, aunque el operador intuitivamente abrió el giro durante la grabación, el wp registrado quedó donde estaba el morro. Al reproducir, la IA sigue ese wp con el morro pero las ruedas traseras barren hacia adentro de la curva — exactamente el sesgo observado.

Conclusión preliminar: regrabar la ruta NO resuelve este problema (el recording mismo tiene el sesgo de captura). Es **estructural del control de steering**, requiere compensación geométrica explícita en el playback.

Para el paper: una vez confirmado con análisis del CSV, esto es una de las contribuciones técnicas concretas del framework — identificar que eAI no parametriza por geometría del vehículo y compensar en la capa de playback. Posicionar como **caso de estudio del corredor 2D**: "el corredor no es solo tolerancia lateral cómoda, es compensación geométrica necesaria para vehículos largos."

Próximos pasos (cuando se retome):

1. Análisis del último CSV de AI logging filtrado por curvas pronunciadas — confirmar sesgo direccional.
2. Si confirmado, implementar ofset de lookahead proporcional a $\text{wheelbase} \times \text{local_curvature}$ como parte del corredor.
3. Probar mismo recording con Hatchback vs Bus, comparar lateral_dev_m — diferencia debería ser proporcional al ratio de wheelbases.

El orden de las capas importa — corredor como guardarraíl, no como parche (*insight 2026-05-24, el corredor*):

Al implementar el corredor (rieles laterales, deadband espacial en steering), Sonom4n señaló:

Esto es una observación arquitectónica fundamental: el corredor NO es la solución gruesa que arregla la trayectoria, es la **última capa de garantía** sobre un sistema ya alineado. Cada capa supone

que las anteriores hicieron su trabajo:

Capa	Resuelve	Si falla esta capa, las siguientes...
Recording humano	Forma de la trayectoria, intención	Sin recording bueno, el corredor estaría centrado en una línea zigzagueante. Las paredes mismas serían inestables
Lookahead adaptativo	Suavidad en curvas (no cortar por adentro)	Sin lookahead adaptativo el bus apuntaría adelante en curvas, tocando pared interior constantemente
Modelo físico predictivo	Llegar exacto a paradas, modular freno	Sin modelo físico, el bus oscilaría lateralmente durante la deceleración, tocando ambas paredes
Corredor (rieles)	Tolerancia lateral, guardarraíl para excepciones	(capa final, no hay siguientes)

Implementado al principio (hipotético): el corredor habría funcionado como **vendaje sobre síntomas** — el bus se mantendría en ruta pero con violencia visual constante (traqueteo permanente contra las paredes). Peor aún: hubiese **escondido los problemas reales** de las capas anteriores, deteniendo la iteración antes de tiempo.

Implementado al final (este caso): el corredor casi no se activa. Es guardarraíl para los pocos puntos donde aún hay roces (rotonda Chernobyl, curva pronunciada) y para condiciones no testeadas (peso, lluvia, vehículo nuevo sin recalibrar). El AI logger CSV va a mostrar mayormente `_inC` (dentro del corredor) y solo ocasionalmente `_outC` (corrigiendo).

Principio general — buena arquitectura de control por capas:

Si en producción ven el corredor trabajando constantemente (mucho `_outC`), no significa que "el corredor falla". Significa que el RECORDING o el LOOKAHEAD ADAPTATIVO están dejando pasar problemas estructurales. La solución no es endurecer el corredor — es volver a las capas tempranas a investigar qué pasó.

Para el paper: este es un capítulo arquitectónico fuerte. Muchos modders intentan resolver problemas de tracking poniendo "corrección agresiva" como parche. La lección del framework es la contraria: **la corrección agresiva tardía esconde problemas estructurales tempranos**. La trayectoria estable nace del recording, no del control. El control solo garantiza márgenes ante imprevistos.

Vale como sección del Capítulo 4 (framework genérico) o como cierre arquitectónico del paper. Conecta directo con el principio "intención sobre realismo impuesto" — el corredor preserva la INTENCIÓN del operador en el espacio (la trayectoria amplia y suelta) sin imponer correcciones que la deformen.

Extensión del insight — corrección externa fuerte = el sistema no produce el comportamiento, lo simula (Sonom4n, 2026-05-24 continuación):

Sonom4n conectó el corredor-sin-capas-previas con el primer intento de la primerísima sesión, donde el bus se movía vía `SetPosition` por script (motor apagado, ruedas inmóviles, sin animaciones del conductor). Ese método "funcionaba" — el bus llegaba a destino — pero se veía como un sprite teleportándose por una línea, no como un vehículo. Estaba siendo **arrastrado por código**, no manejado.

Es la misma anti-patrón en dos planos distintos:

- **Plano longitudinal** (primer método): `SetPosition` forzaba el

avance ignorando motor, físicas, animaciones. El bus se movía por una línea pero el sistema vehicular completo estaba inerte.

- **Plano lateral** (corredor sin capas previas hipotético): el corredor forzaría la posición lateral ignorando si el steering del NPC, las físicas de inclinación del chasis y la geometría del giro tienen sentido coherente. El bus iría por la trayectoria pero las ruedas trazarían arcos imposibles, el chasis no se inclinaría, Boris haría inputs de steering erráticos.

Principio general: cuanto más fuerte la corrección externa, más se nota que el sistema no está produciendo el comportamiento por sí mismo. Los humanos detectamos esto aunque no sepan articular por qué se siente raro — es la diferencia entre el auto de GTA III (script puro, sin físicas) y el de Forza (físicas reales). Los dos llegan al destino, uno se siente vehículo y el otro se siente sprite. La señal de "naturalidad" emerge de la **consistencia interna del sistema**, no de la corrección de su output.

El framework apuesta a lo opuesto explícitamente: dejar que eAI corra su lógica completa (motor, animaciones, transmisión, animaciones del NPC, sonidos, gear shifts), interceptar solo el output **mínimo necesario, subordinar pero no reemplazar**. El corredor encaja en este principio porque es **moduladora del target del steering**, NO un override del steering en sí. eAI sigue calculando su propio input de steering, sigue dando animación al conductor, sigue todo. Solo le decimos a qué heading apuntar.

Tabla — qué intercepta el framework vs qué deja correr:

Sistema	Quién controla	Por qué
Motor (encendido, RPM)	eAI / Car vanilla	Sonido, calor, fuel consumption
Animaciones del conductor	eAI vanilla	Inmersión, IK del volante
Animaciones del vehículo (suspensión, lights)	DayZ engine	Inclinación en curvas, frenos
Transmisión (gear shifts)	Nuestro AT (en CarScript modded)	Único caso donde reemplazamos, porque eAI hardcodea FIRST
Steering target heading	Framework	Lo único que ata el bus a la ruta
Throttle / brake target	Framework (predictivo)	Necesario para precisión en paradas

Cada fila donde "controla eAI/vanilla" es una decisión consciente de **dejar que el sistema produzca el comportamiento por sí mismo**. Cuando se ven los videos del bus en operación, lo que se "ve real" NO viene de nuestro código — viene de todo lo que DEJAMOS hacer al motor y a eAI. Nosotros solo proveemos el "norte".

Para el paper: esta tabla es probablemente el capítulo más fuerte para vendar la arquitectura. Muestra que el framework no es "un controlador AI nuevo" sino "un mínimo set de hooks sobre sistemas existentes". El esfuerzo de identificar **qué dejar correr solo** fue tanto o más importante que el de escribir el código que sí interceptamos.

Sesión 2026-05-24 — Cierre de v1.0 (corrida canónica completa)

Esta sesión cerró el desarrollo de v1.0 del framework con la generación de la **ruta canónica de ida** validada empíricamente. Compila todos los fixes técnicos y principios articulados durante la sesión, en orden cronológico de descubrimiento.

5.6 — Corredor: tres iteraciones (deadband fallido, Stanley con bug, Stanley correcto)

El corredor (rieles laterales para constreñir la trayectoria del bus) pasó por tres iteraciones, cada una destrabó una limitación de la anterior. Documentamos las tres porque la progresión es instructiva.

Iteración 1 — Deadband espacial absoluto (FALLIDO)

Primera implementación: ancho de corredor fijo (`CORRIDOR_HALF_WIDTH=1.05`). Si el bus está dentro del corredor (`|offset_lateral| < ancho/2`), el target yaw es el heading del segmento del recording (mantener rumbo). Si está fuera, pure pursuit normal lo trae al centro.

Resultado empírico: zigzag amplio a alta velocidad. El bus dentro del corredor podía derivar libremente (heading levemente desviado, sin corrección). Al tocar pared, pure pursuit fuerte de golpe → volantazo

- inercia del chasis → cruza al otro lado → repeat.

Banda	dev avg con deadband	% outC
Lento (<20 km/h)	1.00m	3.6%
Medio (20-40)	0.95m	20.8%
Rápido (≥40)	1.32m	8.4%

Lección: el deadband absoluto en un control con momentum produce oscilaciones. Funciona en sistemas estáticos (termostatos), falla en sistemas dinámicos (vehículos a 50+ km/h).

Articulación del operador que llevó al refactor:

El operador con teclado hace tap-tap-tap a frecuencia de frame, no volantazos sostenidos. El chasis actúa como filtro pasa-bajos sobre los pulsos, integrándolos en yaw rate suave. **Control proporcional continuo a alta frecuencia**, no escalonado.

Iteración 2 — Stanley simplificado con bug de signo

Reemplazo del deadband por **Stanley controller simplificado**:

```
targetYaw = segmentHeading - atan2(K * lateralOffset, velocity)
```

Sin deadband. Corrección proporcional al offset y atenuada por velocidad (divisor `v_ms` reduce la corrección a alta velocidad, exactamente la asimetría que queríamos).

Bug crítico encontrado en producción: el bus se autodirigió al agua en 80 segundos.

Causa: convención de signo del cross product invertida. Mi código:

```
cross = AB.x * AP.z - AB.z * AP.x
```

En sistema left-handed de DayZ (X este, Y arriba, Z norte), la fórmula correcta es:

```
cross = AB.z * AP.x - AB.x * AP.z
```

Con el signo invertido, cuando el bus estaba a la derecha del segmento, `m_CorridorLateralOffset` salía NEGATIVO. Stanley corregía al lado contrario, alejándolo más. Se acumulaba exponencialmente.

Lección — gotcha del paper: las convenciones de cross product en sistemas left-handed vs right-handed no son intercambiables. Si el sistema de coordenadas del motor es ambiguo (DayZ no lo documenta explícitamente), validar empíricamente con un caso conocido:

Iteración 3 — Stanley con signo corregido (FUNCIONA)

Tras el fix de signo, Stanley funciona perfectamente. Resultado en corrida canónica completa (24.92 km, 14 paradas, 31 minutos):

```
dev lateral:    avg 1.00m  median 0.88m  p95 2.22m  max 3.18m
|steer|:        avg 0.026  max 0.593
Corredor:      _inC 94.5%  _outC 1.7%
```

Mejora vs iteración 1 con deadband: `_outC` bajó de 8.9% a **1.7%** (5x más estable). En una ruta 5x más larga (7004 wps vs 1766).

K modulado por curvatura local — LECCIÓN NEGATIVA

Intento siguiente para reducir el residual en curvas a alta velocidad (8.9% de samples con dev > 2m): modular K de Stanley en función de la curvatura local (K=1.0 en recta, K=2.5 en curva pronunciada).

Resultado: peor que K=1.0 fijo. Detalles:

Métrica	K=1.0 fijo	K modulado
dev avg global	0.98m	1.01m
dev avg rápido (≥40)	1.14m	1.19m
Rotonda Chernobyl dev avg	0.65m	0.92m
% samples dev>2m	8.9%	10.2%

Diagnóstico: la modulación falló en lo opuesto a lo esperado.

- En la **rotonda Cherno** (curva cerrada y corta), la métrica de curvatura cumulativa la detecta correctamente → K sube a 2.5 → **sobre-corrige donde ya iba bien** (de 0.65m a 0.92m).
- En **curvas largas a alta velocidad** (zonas problemáticas reales), la curvatura local cumulativa NO supera el umbral → K queda en 1.0 → **no actúa donde sí se necesita**.

Lección del paper — métrica equivocada: el problema en curvas a alta velocidad no es la geometría (curvatura sola), es la **fuerza centrífuga** = $\text{curvatura} \times \text{velocidad}^2$. El atenuador $1/v$ que ya integra el Stanley clásico provee la modulación correcta sin métrica adicional. Cualquier modulación adicional debe usar **(curvatura $\times v^2$)** como input, no curvatura sola.

Decisión final: revertir a K=1.0 fijo. El experimento del K modulado queda documentado como contribución de investigación negativa.

5.7 — Fix humano-driver: eAI hereda PlayerBase

El problema arquitectónico

El framework necesita un mecanismo para que el operador pueda **tomar el bus** durante una corrida IA — para grabar tramos nuevos, corregir en vivo, o continuar grabaciones (ver sección hand-off). Implementación inicial:

```
override void OnInput(float dt) {
    super.OnInput(dt);
    ...
    Human driver = CrewMember(0);
    if (driver && PlayerBase.Cast(driver)) {
        return; // humano al volante, service se hace a un lado
    }
    srv.ApplyBusInput(this, dt);
    ...
}
```

El bug

`PlayerBase.Cast(driver)` devuelve **true también para Boris** (el eAI). En DayZ, los NPCs eAI heredan de `PlayerBase` para reutilizar movimiento/animaciones/inventario. Mi check detectaba a Boris como "humano" y el service se hacía a un lado **cuando NO debía** → el bus no recibía inputs → rodaba solo por inercia a ~1.5 km/h.

El fix

```
Human driver = CrewMember(0);
if (driver) {
    PlayerBase realPlayer = PlayerBase.Cast(driver);
    if (realPlayer && realPlayer.GetIdentity()) {
        return; // player REAL (tiene PlayerIdentity)
    }
}
```

La diferencia técnica entre player real y eAI: **solo el player tiene `PlayerIdentity`**. eAI puede ser `PlayerBase` pero `GetIdentity()` devuelve null.

Gotcha para el paper

Documentar en Capítulo 5 (Gotchas de Enforce):

Este patrón aparece también en `BroadcastDistances` del propio `BZBusService`: el filtro de "jugadores válidos" usa `!player || !player.GetIdentity()` como rechazo. La convención está, no está documentada explícitamente.

Implicancias positivas inesperadas

Una vez funcional, el fix habilita una propiedad emergente:

Hand-off operador ↔ IA en cualquier momento de una corrida. El operador puede subirse al bus mientras la IA maneja, manejar él manualmente un tramo, bajarse y la IA retoma. Convierte el recording de un acto único en un **proceso incremental, iterable y colaborativo**.

Casos de uso habilitados:

- Grabaciones largas en sesiones cortas (terminar la ruta otro día)
- Iteración por sub-tramos (regrabar solo lo que está mal)
- Colaboración multi-operador (distintas personas graban distintos tramos)
- Hot-patching en vivo durante corrida IA (corregir en el momento)
- Canal manual de ILC (complementario al automático del v2.0)

5.8 — Spawn robusto: pre-roll + ValidateSpawn con criterio de movimiento

El problema

`ValidateSpawn` (auto-retry del spawn cuando el bus no se mueve 2m en 5s) entró en bucle infinito en producción. Boris se sentaba pero el bus no aceleraba lo suficiente — quedaba a 1.5 km/h, recorría 1.56m en 5s, `ValidateSpawn` disparaba retry, repeat.

Causas combinadas:

1. **Race condition:** motor encendiéndose, driver acomodándose, throttle aplicándose — todo simultáneo en el primer Tick (500ms post-spawn). Si cualquier pieza no estaba lista, el bus no respondía.

2. **Recording leading silence:** los primeros samples del CSV están

con `targetSpeed=0` (operador quieto en la terminal). La cruise control aplicaba throttle bajo, bus aceleraba lento.

3. **Métrica equivocada:** ValidateSpawn medía **distancia recorrida**

(≥ 2 m). La métrica correcta es **bus en movimiento** ($\text{kmh} > 0.5$): si el bus rueda, el driver está sentado y la cadena throttle→motor→ruedas funciona, aunque lento.

Fix 1 — Pre-roll de 3 segundos

Implementación:

```
private void SpawnBus() {
    ...
    m_CachedBrake = 1.0; // brake aplicado desde el primer frame
    m_PreRollEndTime = GetGame().GetTickTime() + 3.0;
    ...
}

// En Tick():
if (GetGame().GetTickTime() < m_PreRollEndTime) {
    if (!bus.EngineIsOn()) bus.EngineStart();
    SetCachedInput(0, 0, 1.0); // brake aplicado
    return; // sin DriveTowards
}
```

Resultado: bus se mantiene 3 segundos con freno aplicado mientras se estabiliza, después arranca normalmente.

Fix 2 — ValidateSpawn por movimiento, no por distancia

```
void ValidateSpawn() {
    if (!m_Bus) return;

    // En parada (m_AtStop) = OK
    if (m_AtStop) {
        m_SpawnAttempt = 0;
        return;
    }

    // En movimiento (kmh > 0.5) = OK
    Car bus_v = Car.Cast(m_Bus);
    if (bus_v && bus_v.GetSpeedometerAbsolute() > 0.5) {
        m_SpawnAttempt = 0;
        return;
    }
}
```

```
// Solo si parado lejos de stop = posible falla
float distFromSpawn = vector.Distance(m_Bus.GetPosition(), m_SpawnInitialPos);
if (distFromSpawn < SPAWN_VALIDATION_MIN_DIST) { ... retry ... }
}
```

Resultado: el bus arranca limpio aunque la grabación tenga "leading silence". El primer wp tiene targetSpeed=9 km/h y el bus acelera gradualmente — antes ValidateSpawn lo terminaba antes de llegar a 2m, ahora lo deja vivir.

Lección del paper — diseñar tests con el criterio correcto

5.9 — Perspectiva de cámara: variable cognitiva del operador

Observación empírica del operador durante la sesión:

Hipótesis articulada

La perspectiva de cámara durante la grabación **modula el estilo de manejo del operador**, no por elección consciente sino por cognición:

	3ra persona	1ra persona
Campo visual	Panorámico, ves más allá del bus	Limitado a lo que un humano real vería
Anticipación	Spotteas curvas a 100m	Solo lo visible por la ventanilla
Referencia espacial	El bus es un sprite a ubicar	Salpicadero, espejos, morro = referencia precisa
Sensación de masa	Casi nula	Sentís altura y peso del bus
Comportamiento típico	Arcade, rápido, "fluye"	Cauto, anticipado, frena antes

Validación empírica

Comparativa del mismo operador grabando dos tramos consecutivos con distinta perspectiva:

Métrica	3ra persona (Kamenka→Cherno)	1ra persona (Cherno→Doliny)		
	steer\	avg banda rápida (≥40 km/h)	0.162	0.107 (-34%)
Cambios de signo del steering	5.2/min	4.5/min		

Conclusión: 1ra persona reduce el input de steering en alta velocidad un 34%. El operador conduce naturalmente más estable, sin volantazos, sin esfuerzo consciente — la perspectiva lo induce.

Recomendación operativa para el manual

Variante del operador inexperto (paradoja contraintuitiva)

El operador con menos rutina sobre un tramo específico maneja **más conservador automáticamente**: no tiene memoria muscular, no anticipa de memoria, frena antes, deja más márgenes. Resultado: **el recording es más seguro**.

Para el paper:

5.10 — Corredor grabado vs corredor calculado (slip angle)

Insight de Sonom4n articulado con dibujo conceptual (preservado en assets del paper):

Estado actual (corredor calculado)

El corredor se calcula matemáticamente desde el recording de línea central: para cada segmento entre dos waypoints consecutivos, la "tapa" perpendicular del corredor es perpendicular al heading de ese segmento. Aproximación geométrica plana.

Limitación geométrica identificada

Un vehículo en una curva NO está perfectamente alineado con la dirección en la que se mueve. Hay un **slip angle** — el chasis apunta su nariz un poco más adentro de la curva de lo que su velocidad lineal indica. En cualquier instante de una curva:

- En recta: ruedas izquierda y derecha perpendiculares a la dirección
→ la "tapa" del corredor es perpendicular al segmento (mi cálculo actual)
- En curva: el chasis está rotado vs la trayectoria → la "tapa" del corredor es **diagonal**, no perpendicular
- En transiciones: la diagonal cambia → forma "torcida" del corredor

El cálculo matemático del corredor (basado en perpendiculares a la línea central) **NO captura esa diagonal**. Subestima la complejidad geométrica real del vehículo en movimiento.

Approach v2 propuesto — Recording de corredor literal

Si el PathLogger capturara la **posición de las ruedas** (delantera- trasera, izquierda-derecha) en lugar de solo el centroide del Car, el "corredor" sería la envoltura real de las trayectorias de las ruedas. En curvas, eso captura naturalmente:

- Geometría asimétrica (rueda exterior viaja más distancia)
- Slip angle implícito (la diagonal de la tapa)
- Off-tracking del eje trasero
- Forma del "embudo" en transiciones de entrada/salida de curva

Analogía Bollinger

Articulación del operador: el corredor en su versión v2 sería análogo a las **bandas de Bollinger** en finanzas — bandas dinámicas cuyo ancho varía con la volatilidad del precio. En nuestro caso, el "ancho del corredor" variaría según la geometría dinámica del vehículo en cada punto. No es media móvil pero el concepto de **márgenes adaptativos en lugar de fijos** es transferible.

Para el paper

Roadmap v2: implementar PathLogger extendido que capture posiciones de cada rueda (4 puntos por sample en lugar de 1 centroide). Eso habilita corredor v2 y captura naturalmente el comportamiento geométrico real del vehículo. Es una **extensión limpia del recording**, no requiere cambios al control.

Limitación honesta del corredor v1 actual: funciona bien para vehículos de wheelbase corto-medio (Hatchback, Sedan). En vehículos largos (bus, V3S), las curvas pronunciadas exponen el desfase geométrico — el bus tiene componente de slip angle no capturado.

5.11 — Bidireccionalidad como caso de uso fuerte

Observación del operador durante la sesión:

Redefinición del corredor

Antes: el corredor era "guardarraíl para excepciones" (curvas problemáticas, condiciones no testeadas).

Ahora: el corredor es **prerequisito técnico para servicios multi-direccionales**. Sin corredor activo, dos vehículos en sentidos opuestos sobre el mismo trazado pueden invadir el carril contrario en curvas — colisión frontal garantizada.

Implicancias técnicas

1. El espejo automático del recording NO sirve. Invertir

matemáticamente la ida (waypoints en orden reverso) produce una trayectoria que pasa por **el mismo carril físico** que la ida. La perpendicular geométrica no resuelve esto: el "otro carril" puede estar fuera del camino, en banquina o cuneta.

1. La vuelta tiene que ser un recording independiente. El operador

maneja físicamente por **su carril** desde el destino hacia el origen. Resulta una trayectoria paralela a la ida pero ofseteada lateralmente ~3.5m (ancho típico de carril). Mismo principio que un chofer humano de ida vs uno de vuelta — son dos personas distintas pasando por dos líneas distintas.

1. El corredor + Stanley garantizan no-invasión del carril contrario.

Las desviaciones laterales del bus se mantienen en $\pm 1\text{-}2\text{m}$ (medido empíricamente en corrida canónica). Como un carril mide ~3.5m, esa tolerancia es **menor que el ancho del carril** — el bus puede oscilar dentro de su carril sin invadir el contrario.

Para el paper

Esto cambia el discurso del corredor:

Y para los **casos de uso del Capítulo 6** (logística, patrullas, convoyes, ladrones de vehículos) — todos requieren que los vehículos respeten sus carriles cuando se cruzan. El corredor + Stanley es la primitiva técnica que habilita todo eso.

5.12 — Bus funciona sin driver: descubrimiento arquitectónico

Observación durante producción: las últimas paradas de una corrida, Boris se despawneó (probablemente por daño acumulado en pequeñas colisiones — el driver no estaba protegido con `SetAllowDamage(false)` mientras que el bus sí). **El bus siguió manejándose normalmente**, con velocidad, giro de ruedas, llegada a paradas, sin sonido de motor.

Por qué funciona

`SetThrottle/SetSteering/SetBrake` son inputs al **Car**, no al driver. El motor de DayZ solo requiere `EngineIsOn() == true` para procesar esos inputs — no requiere un humano en seat 0. El driver aporta:

- Animaciones visuales (brazos al volante)
- Trigger del sonido del motor

Pero NO es necesario para el control físico del vehículo.

Implicación arquitectónica

El framework no depende de eAI para el control. Solo lo usa como:

- Capa visual (animaciones del conductor)
- Capa de pathfinding inicial (waypoints del eAIGroup)
- Audio (sonido del motor)

Si en el futuro queremos:

- **Buses fantasma** (sin NPC visible, p.ej. para minimizar carga de IA)
- **Drones rodantes** (sin operador visible)
- **Resiliencia** (driver muere o se cae → servicio continúa)

el código actual ya lo soporta. Es **subordination architecture llevada al extremo** — eAI puede morirse y el control sigue.

Para el paper

Validación inesperada del principio "subordinar pero no reemplazar". El framework SÍ sobrescribe los inputs de control de eAI, pero NO depende de que eAI esté vivo para funcionar. Es **acoplamiento unidireccional**: nosotros leemos lo que eAI provee (animaciones, pathfinding), pero el control mismo es nuestro.

5.13 — Markers como etiquetador empírico en vivo

Mecanismo

El operador puede pulsar `NUMPAD -` durante una corrida IA. Eso marca el próximo sample del CSV con `is_marker=1`. Diseñado originalmente para anotar eventos negativos (roce, frenazo raro), resultó útil también para **anotar aciertos** (curva bien tomada, maniobra precisa).

Resultado empírico — corrida canónica completa

El operador pulsó 9 markers durante 31 minutos de corrida. Análisis post-hoc:

#	t (s)	wp	kmh	dev	Tipo
1	135	537	35.6	0.69m	acierto
2	199	773	63.8	1.32m	acierto (curva rápida)
3	423	1619	23.7	0.11m	acierto (rotonda con dev cero)
4	520	1985	17.4	0.55m	acierto (maniobra precisa)
5	738	2855	26.6	1.08m	acierto
6	921	3602	15.1	0.68m	acierto
7	1466	5648	36.9	0.41m	acierto (curva rápida con dev baja)
8	1627	6253	17.6	0.61m	acierto
9	1852	7030	0.2	2.45m	único negativo (curva final, freno excesivo del recording)

Ratio aciertos/problemas: 8/1

dev media de los markers de acierto: 0.69m — significativamente mejor que el promedio global de la corrida (1.00m).

Interpretación para el paper

El operador actuó como **etiquetador empírico en vivo** durante la corrida. Su anotación implícita: "estos 9 momentos en 31 minutos merecen atención". Cruzando con datos:

- 8 son momentos donde la IA superó las expectativas del operador (manejó bien algo difícil)
- 1 es momento donde la IA falló las expectativas (frenó de más por el recording)

Conexión metodológica

Esto valida el principio recording-driven con **datos cruzados de dos fuentes independientes**: la anotación humana en vivo + la métrica automática del AI logging. Cuando ambas coinciden en identificar los "buenos momentos", el framework está haciendo lo correcto.

5.14 — Scorecard final del framework (v1.0)

Corrida canónica completa, 2026-05-24 15:25:

Ruta: Kamenka → 13 paradas intermedias → Terminal Svetloyarsk+

Distancia: 24.92 km

Duración: 31.3 min

Vel media: 47.8 km/h

Waypoints: 7004 (3 segmentos con perfiles de operador distintos)

Paradas: 14, todas activadas correctamente

Llegada a terminal: sí (con tramo final regrabado para corrección del freno excesivo del operador en la primera toma)

Métricas de calidad:

- dev lateral avg: 1.00m
- dev lateral median: 0.88m
- dev lateral p95: 2.22m
- dev lateral max: 3.18m
- |steer| avg: 0.026 (volante casi inmóvil – control suavísimo)
- |steer| max: 0.593
- _inC (en corredor): 94.5%
- _outC (corrigiendo): 1.7%

Eventos cruzados con observación humana:

- 8/9 markers del operador en vivo: aciertos (dev media 0.69m)
- 1/9: problema atribuible a freno excesivo del recording original

Progresión histórica del framework

Iteración	dev avg	%outC	Llegada	Lección clave
Sin nada (pure pursuit fijo)	~1.5-2.0m est	—	parcial	el lookahead fijo no escala
Lookahead adaptativo + recording v1	1.22m	10.6%	sí	grabación humana = mejora gratis
Corredor con deadband	1.22m	10.6%	sí	deadband + momentum = zigzag
Stanley K=1.0 (sin signo fix)	bus al agua	—	NO	gotcha de cross product
Stanley K modulado	1.01m	10.2%	sí	métrica equivocada (curvatura sola)
Stanley K=1.0 con signo OK	1.00m	1.7%	sí, 24.92 km	el clásico de literatura, bien calibrado

Cuánto sirvió cada técnica

Atribución estimada de la mejora total (dev global 2m → 1m):

1. **Recording-driven humano** (40%) — la trayectoria viene del

operador, el control no la inventa

- 2. **Stanley con signo correcto** (25%) — corrección continua proporcional, no escalonada
- 3. **Fix humano-driver v2** (15%) — sin esto el bus no respondía con Boris al volante (regresión funcional crítica)
- 4. **Pre-roll + ValidateSpawn fix** (10%) — spawn robusto, no entra en bucle de retries
- 5. **Lookahead adaptativo** (5%) — suaviza el control en curvas
- 6. **Modelo físico de freno + stopDuration 2s** (5%) — llegadas precisas a paradas

Las dos primeras suman 65%. Lo más impactante del framework es **capturar la intención del operador y aplicar corrección proporcional sin volantazos**. El resto es ingeniería de soporte.

5.15 — Tabla "quién controla qué" — arquitectura definitiva del framework

Subsistema	Quién controla	Por qué
Motor (encendido, RPM, sonido)	eAI / Car vanilla	Inmersión, físicas reales
Animaciones del conductor (IK volante, mirada)	eAI vanilla	Movimiento natural del NPC
Animaciones del vehículo (suspensión, lights, brake lights)	DayZ engine	Inclinación en curvas, físicas
Transmisión / shifting de gear	Framework (CarScript modded)	eAI hardcodea FIRST → tenemos que reemplazar
Steering target (a dónde apuntar)	Framework (Stanley sobre corredor)	Lo único que ata el bus a la ruta
Throttle / brake target	Framework (control predictivo + cruise)	Necesario para precisión en paradas
Pathfinding global (waypoints)	eAI vanilla	Le seteamos waypoints, eAI navega
Pathfinding reactivo (evitar obstáculos puntuales)	No implementado (roadmap v2)	Sería capa adicional
Detección y reacción a otros vehículos	No implementado (roadmap)	Choque pasivo por ahora
Daño y resiliencia del vehículo	<code>SetAllowDamage(false)</code>	Bus invulnerable durante servicio

Subsistema	Quién controla	Por qué
Daño y resiliencia del driver	Pendiente (sin proteger)	Boris muere por colisiones, deberíamos protegerlo

Para el paper: esta tabla es probablemente el capítulo más fuerte para vender la arquitectura. Muestra que el framework no es "un controlador AI nuevo" sino "un mínimo set de hooks sobre sistemas existentes". El esfuerzo de identificar **qué dejar correr solo** fue tanto o más importante que el de escribir el código que sí interceptamos.

5.16 — Roadmap inmediato (v1.1)

Tareas técnicas pendientes con prioridad:

1. **Fix invulnerabilidad del driver:** agregar `m_Driver.SetAllowDamage(false)` después de spawnearlo. Y monitor en Tick para respawnear Boris si se detecta que murió o desapareció (no solo "se bajó del bus" como actualmente).
1. **Grabación de vuelta** (Terminal → Kamenka): recording independiente por carril opuesto. Probará empíricamente el caso de uso de bidireccionalidad descrito en 5.11.
1. **Multi-rutas en BZBusService:** extender el servicio para cargar N rutas (ida + vuelta + posibles desvíos al interior) y spawnear N buses, uno por cada ruta. Refactor del JSON config a array.
1. **Parámetro `FailsafeMode` en config JSON:** `"FAITHFUL"` (default, reproduce todo del recording sin asistencia) o `"CONSERVATIVE"` (activa safety nets: anti-stall si bus parado lejos de stop, override de freno excesivo, etc.). Para servidores en producción real que necesitan robustez ante operadores menos meticulosos.
1. **Prueba multi-vehículo:** validar Capítulo 3 (herencia CarScript) con vehículos vanilla (Hatchback, V3S, Sedan) y modded del Workshop. Misma ruta, distintos vehículos, sin tocar código. Predicción: vehículos chicos y ágiles tendrán dev avg significativamente menor que el bus.

5.17 — Roadmap v2.0 (siguiente versión mayor)

1. **Corredor v2** — PathLogger extendido que captura posiciones de las 4 ruedas. Habilita corredor con slip angle implícito (sección 5.10). Más fiel para vehículos largos.
1. **Iterative Learning Control (ILC) automatizado** — el framework compara cada corrida IA contra el recording, calcula error, ajusta parámetros internos (Stanley K, MAX_BRAKE_DECEL, etc.) automáticamente. Canal complementario al manual del hand-off.
1. **Pathfinding reactivo** — usar `m_PathFinding` de eAI a pie

(el sistema navmesh-based, más sofisticado que el vehicular) para micro-correcciones en zonas críticas marcadas en el JSON. Combinar con velocidad baja para precisión sub-metro en rotondas o estacionamientos.

1. **Experimento de peso** — 5 NPCs + baúl lleno en el bus. Validar

empíricamente la propiedad de invariancia al peso del control predictivo de freno (predicha por la fórmula $a = u^2 / (2s)$ que es masa-agnóstica).

1. **Recording auto-trim** — al convertir CSV → JSON, descartar

automáticamente los samples leading y trailing con velocidad < 1 km/h ("silence trim" como en audio). Evita problemas de spawn lento y de bus parado al final de la ruta.

5.18 — Conclusión narrativa de la sesión

Esta sesión cerró v1.0 del framework con la generación del **artefacto canónico** (ruta de ida de 24.92 km, 14 paradas, 7004 waypoints) y la validación empírica de **todos los principios articulados en sesiones previas**:

- "Intención sobre realismo impuesto" — los tres segmentos con perfiles distintos del operador (agresivo / conservador 1ra / regrabado) reproducen fielmente cada estilo
- "Subordinar pero no reemplazar" — bus funcionó sin driver, eAI puede morir y el control sigue
- "Recording-driven" — la mejor manera de mejorar el bus es regrabar los tramos problemáticos, no tunear el código
- "Capa final hace menos trabajo, no más" — el corredor se activa solo en 1.7% de los samples, las capas tempranas resuelven el 95%+ del trabajo

La sesión también descubrió bugs y limitaciones que no habíamos articulado antes:

- **Cross product en sistema left-handed**: convención no documentada por el motor, requiere verificación empírica
- **eAI hereda de PlayerBase**: el discriminador correcto es `GetIdentity()`, no `PlayerBase.Cast()`
- **Recording leading silence**: el operador grabando inicia con velocidad cero, hay que tratarlo (trim o anti-stall)
- **Pre-roll del spawn**: el playback debe arrancar cuando el sistema está listo, no inmediatamente post-creación de entidades

Y articuló nuevos principios para el paper:

- **Markers como etiquetador empírico en vivo** — el operador anota aciertos y fallos durante la corrida, cruzando con datos métricos

- **Perspectiva de cámara modula el estilo** — 1ra persona reduce steering 34% en alta velocidad, sin esfuerzo consciente
- **El operador inexperto graba más seguro** — paradoja contraintuitiva con valor concreto para producción

El cierre del paper se sostiene sobre estos pilares.

Sesión 2026-05-24 PM — Generalización a otros vehículos y el espectro fidelidad/naturalidad

Tarde de la misma sesión, foco distinto: validar Capítulo 3 (herencia CarScript) empíricamente con un vehículo radicalmente distinto al bus (Land Rover Defender de Expansion), y articular **los límites filosóficos del framework**.

5.19 — Validación empírica del Capítulo 3: herencia automática con Expansion_Landrover

Setup

- **Vehículo:** Expansion_Landrover (Defender 110 verde militar)
- **Cambios al código framework:** ninguno. Solo configuración por JSON.
- **Tres tomas grabadas en escalada de control:**

Toma	Estilo	Vel avg	Vel max	Gear max	Duración	Hallazgos
T1 stress-test	Agresivo Stry Sobor→Zenit	60.5 km/h	100.0	6ta (gear 7)	4.9 min	Usó toda la capacidad del vehículo, 1 marker
T2 moderada	Inverso Zenit→Stry Sobor	46.3	96.7	5ta (gear 6)	6.3 min	"Vivir en 30-60", sin 6ta
T3 controlada	Inverso con paradas técnicas	38.5	77.5	5ta	7.6 min	61.3% sin throttle, 4 markers (3 curvas 90° + terminal)

Comportamiento del framework — sin tocar código

Ninguno de los cambios fue al código del control. Todos fueron configuración:

- VehicleClass = "Expansion_Landrover" en JSON
- MaxGear = 7 en JSON (vs 6 del bus) — usar 6ta marcha
- Attachments (lista de classnames) en JSON — ruedas/batería/bujías/etc. del Land Rover

- Recording independiente (tres tomas distintas)

El framework **no preguntó qué vehículo era**. Identificó que era un `CarScript` (herencia automática), aplicó Stanley + corredor + control predictivo + pre-roll + ValidateSpawn + todo lo demás, idéntico al bus.

Validación del principio del Cap 3:

Empíricamente confirmado: bus + Land Rover comparten 100% del código de control. La diferencia entera son **4 campos del JSON** (VehicleClass, MaxGear, Attachments, recording).

5.20 — Vehicle profile como abstracción primaria del framework

La sesión obligó a hacer **explícitos** parámetros que antes eran implícitos (hardcodeados para el bus). Esto cristalizó el concepto de **vehicle profile**: el conjunto mínimo de datos que distingue un vehículo del siguiente, en términos que el framework consume.

Campos del vehicle profile (v1.0 actual)

```
{
  "VehicleClass":  "Expansion_Landrover",
  "DriverClass":   "eAI_SurvivorM_Boris",
  "MaxGear":       7,
  "Attachments":   ["expansion_landrover_wheel", ...],
  "Waypoints":     [...]
```

Campos en roadmap v1.x

Validados conceptualmente pero no implementados en v1.0:

```
{
  "STANLEY_K":      1.0,    // ganancia lateral (bus=1.0, vehiculos rapidos > 1.0)
  "STEERING_SCALE": 1.0,    // cap del steering (bus=1.0, wheelbase corto < 1.0)
  "MAX_BRAKE_DECEL":50,    // capacidad de freno (bus=50, vehiculo liviano < 50)
  "MASS_FACTOR":    1.0,    // escala el g·sin(θ) del modelo predictivo
  "FailsafeMode":   "FAITHFUL" // o "CONSERVATIVE" con anti-stall y safety nets
}
```

Lectura arquitectónica

El framework v1.0 expone **lo justo para un vehículo nuevo**: clase, driver, max gear, attachments, recording. Esos 5 campos son **el contrato mínimo** entre el operador y el framework. Si los proveés, funciona.

La sesión validó que con perfiles **radicalmente distintos** (bus ~10000 kg, 5-6m wheelbase, max 80 km/h vs Land Rover ~2000 kg, 2.7m wheelbase, max 100+ km/h), el framework opera. Hay calidad

residual distinta — pero **opera**. Eso es Capítulo 3 validado empíricamente.

Para el paper este es el caso de estudio más sólido: misma sesión, mismo código, mismo operador, vehículos casi diametralmente opuestos. Ambos llegaron a destino. La adaptación viene de:

- La **geometría implícita en el recording** (el operador grabó cada vehículo a su ritmo natural)
- Los **4 campos del vehicle profile** (clase, max gear, attachments)
- Cero cambios al control

5.21 — Trader files como fuente de classnames de attachments

Gotcha encontrado

El primer intento de configurar attachments del Land Rover usó `Expansion_Landrover_Wheel` (PascalCase, asumido análogo al `ExpansionBusWheel` del bus). Resultado: vehículo spawnó **desnudo** porque los classnames de Expansion son **case-sensitive minúscula** — `expansion_landrover_wheel`, no `Expansion_Landrover_Wheel`.

Solución elegante

Los trader files de Expansion (`mpmissions/<map>/expansion/traderzones/*.json`) listan TODOS los classnames vendibles, incluidos attachments de vehículos. `Taller_Ratnoye.json` resultó tener:

```
expansion_landrover_wheel
expansion_landrover_hood
expansion_landrover_trunk
expansion_landrover_driverdoor
expansion_landrover_codriverdoor
expansion_landrover_left
expansion_landrover_right
```

Patrón para vehículos nuevos

Para el paper

Anotar como gotcha del Capítulo 5: **classnames case-sensitive**. Expansion usa snake_case minúscula, vanilla usa PascalCase. Mezclar los dos es común y silencioso (DayZ no avisa, devuelve null).

5.22 — Paradas técnicas: extensión semántica del mecanismo de stop

Definición

Originalmente las paradas (`isStop=true`) eran **paradas de pasajeros** del bus (Kamenka, Chernobyl, etc.). La sesión validó un uso semántico distinto del mismo mecanismo: **paradas técnicas**.

	Parada de pasajeros	Parada técnica
Propósito	Subir/bajar gente	Forzar aproximación a velocidad cero a una coord específica
Marcado	NUMPAD 4 en el cartel	NUMPAD 4 antes de una curva difícil, encrucijada, maniobra
Nombre en JSON	"Chernogorsk", "Balota", etc.	"Curva 90 #N" o sin nombre
Comportamiento del bus	Frena, espera <code>stopDuration</code> , broadcast a chat, sigue	Frena, espera <code>stopDuration</code> , sin broadcast, sigue
Caso de uso	Bus de transporte	Land Rover en curvas 90°, cualquier vehículo en maniobras

Validación empírica

En la toma 3 del Land Rover, el operador marcó **3 paradas técnicas** antes de curvas 90° pronunciadas. Las paradas técnicas no son "paradas reales" para los usuarios — son **garantías de precisión** para el framework: el operador le dice "acá te bajás a 0, doblás lento, después seguís".

Para el paper

Una propiedad emergente del framework: **el mecanismo de paradas acepta una semántica que excede el caso de uso original (bus)**. El mismo código sirve para "parada de pasajeros" y "parada técnica" — solo cambia la intención del operador al pulsar NUMPAD 4. El framework no necesita saber qué tipo de parada es.

Esto es buena arquitectura: **mecanismo simple + semántica del operador = casos de uso múltiples**.

5.23 — Engine braking en pendiente: validación del "framework reproduce lo que NO se hace"

Observación empírica

Durante la toma 3 del Land Rover, el operador encontró un descenso de 86 metros de elevación en ~530 metros horizontales (~16% de pendiente). En esa sección de **44 segundos**, NO presionó throttle en absoluto — dejó que la combinación de gravedad + engine braking + freno físico moderado controlara la velocidad.

Inicio: `t=67s, pos=(8244, 9070), y=466.9m, kmh=41.8`
Fin: `t=110s, pos=(8774, 9096), y=380.7m, kmh=13.0`
Delta: -86.2m elevación, 44s sin throttle, decel ~6 m/s² promedio

El recording capturó `targetThrottle=0` durante 173 samples consecutivos.

Implicancia conceptual

El framework reproduce **no solo lo que el operador HACE, sino también lo que el operador NO HACE**. Soltar el acelerador es una **decisión activa** del operador. El framework la captura como `throttle=0` y la honra literalmente: durante 44 segundos el bus IA NO va a acelerar, va a dejar que la pendiente lo lleve.

Para el paper

Conecta con principios anteriores:

- "Subordinar pero no reemplazar"
- "Intención sobre realismo impuesto"
- "Capa final hace menos trabajo, no más"

Es validación adicional de los tres a la vez.

5.24 — Alcance y límites del framework: el espectro fidelidad/naturalidad

El espectro de cuatro puntos

Cualquier sistema que "mueve un vehículo de A a B" opera en algún punto de este espectro:

Enfoque	Fidelidad	Naturalidad	Caso de uso
eAI vanilla sin guía	~0% (errático)	media	autos genéricos de background
Recording-driven puro (este framework)	~95%	máxima	bus, vehículos cotidianos, casos "humanos"
Híbrido con snapshot correctivo	~99%	media (sprite-like en momentos críticos)	convoyes precisos, misiones críticas
Snapshot puro (SetPosition)	100%	cero	cinematics, scripts narrativos

Cada enfoque tiene un trade-off **inevitable** entre fidelidad (qué tan cerca llega al objetivo geométrico exacto) y naturalidad (qué tan "real" se ve el comportamiento desde adentro del juego).

El framework v1 vive en el segundo punto

Toda fidelidad viene de **respetar la física del vehículo**. Las desviaciones residuales (dev avg ~1m en bus, predicción <0.5m en Land Rover) son el costo de mantener la naturalidad. Si quisiéramos fidelidad perfecta, perderíamos el "se ve real" que hace que un pasajero virtual lo sienta como un viaje en colectivo.

Subir en fidelidad sin perder naturalidad (v2 micro-tuning)

Hay margen dentro del segundo punto del espectro, sin saltar al tercero:

- **Samples más frecuentes en grabación:** PathLogger a 100Hz (cada 10ms) en lugar de 4Hz. Captura manejo sub-perceptual del operador.
- **Inputs interpolados entre Ticks:** ya `ApplyBusInput` corre cada frame, pero `m_Cached*` cambia solo cada Tick. Interpolan los cached entre Ticks aumentaría fidelidad sin volantazos.
- **Look-ahead temporal del input:** aplicar el input que el operador hubiera aplicado N samples adelante, capturando la "anticipación humana".

Estos son **micro-tunings de discretización temporal**. Suben la fidelidad probablemente al 98%. **Siguen siendo recording-driven.**

Cuándo saltar a otra arquitectura

Si necesitás fidelidad >99% o naturalidad no importa, el segundo punto del espectro no es óptimo. Casos:

- ****Convoyes militares donde TODOS los vehículos deben llegar al checkpoint sin desviarse**** → snapshot híbrido con correcciones forzadas en momentos críticos
- **Cinemáticas donde la cámara está bien definida** → snapshot puro, el sprite no se nota porque la cámara no muestra inputs
- ****Misiones de tipo "el camión llega exacto a este punto a este segundo"***** → híbrido o snapshot

El framework v1 **no es óptimo** para esos casos. Es óptimo para:

- Servicios de transporte (bus, taxi, ambulancia)
- Patrullas con timing relativo
- Convoyes donde "se vean reales" importa más que "lleguen exactos"
- Cualquier vehículo donde un pasajero virtual va adentro

5.25 — Discretización temporal como el umbral real

Sonom4n articuló también: "el humano sí lo puede reproducir" (vos podés manejar el DeLorean a 100 km/h con éxito). Entonces ¿por qué el framework no?

La respuesta no es la física del vehículo. Es la frecuencia de control.

Asimetría fundamental humano vs IA

	Humano operando	IA del framework v1
Frecuencia de input	Sub-perceptual (~100 Hz efectivo)	Tick discreto (~2-4 Hz)

	Humano operando	IA del framework v1
Feedback	Tiempo real (vista, sonido, sensación de inercia)	Solo lee samples grabados, no estado actual del vehículo
Pre-corrección	Anticipa el efecto antes que se vea	Solo reacciona a samples ya grabados
Adaptación	Continua, micro-corrige cada frame	Discreta, espera al próximo Tick

Aritmética del umbral

A baja velocidad, la latencia del Tick (250-500ms) es invisible porque los errores absolutos son chicos:

- Bus a 50 km/h $\times$ 500ms = 7m de "ceguera" entre Ticks $\rightarrow$ dentro del corredor (1m)
- Land Rover a 80 km/h $\times$ 500ms = 11m de ceguera $\rightarrow$ marginal
- DeLorean a 200 km/h $\times$ 500ms = 28m de ceguera $\rightarrow$ catastrófico

El umbral **NO es la velocidad máxima del vehículo**. Es `velocidad  $\times$  latencia_del_control`. Bajar la latencia (v2 micro-tuning) sube el umbral.

Implicancia para el paper

Esto es importante porque articula que **el framework no está "limitado por la física del juego"**. Está limitado por una elección de implementación (Tick de 500ms) que era razonable para el caso inicial (bus en ciudad) y se vuelve insuficiente para casos extremos. La solución no es cambiar de paradigma, es **subir Hz**.

5.26 — Principio rector emergente: "el framework opera en el entorno humano físicamente posible"

Síntesis de los puntos 5.24 y 5.25:

Conclusión del paper a articular

Este principio rector define el alcance del trabajo y honestamente reconoce sus límites. Es uno de los pilares del cierre del paper — no se vende como "solución universal" sino como **solución óptima para un espacio bien definido y empíricamente validado**.

5.27 — Qué decide el framework vs qué decide el vehículo + recording

Observación de Sonom4n al cierre, refinando el principio:

Articula que el framework controla **inputs nominales** (throttle 0-1, steering -1..1, brake 0-1), pero **la conversión de input $\rightarrow$ fuerza física** la decide el motor del juego según el vehículo.

Tabla refinada de quién decide qué

Magnitud	Decide framework	Decide vehículo + motor	Decide recording
Ángulo del volante (input)	sí (Stanley)	—	—
Yaw rate efectivo (resultado)	—	sí (wheelbase, slip, masa)	—
Pedal de throttle (input 0-1)	parcial (cruise mode)	—	parcial (cuando se respeta)
Aceleración longitudinal (resultado)	—	sí (torque curve, gear, masa, fricción)	—
Pedal de freno (input 0-1)	sí (modelo predictivo)	—	—
Deceleración (resultado)	—	sí (frenos, peso, suelo)	—
Gear	sí (AT modded)	—	—

Implicancia: el framework opera en **el espacio de inputs**, no en el espacio de outputs físicos. El operador grabando OBSERVA el espacio de outputs (lo "siente" en el juego) y ajusta sus inputs en consecuencia. Esa traducción intuitiva del humano (sentir output → ajustar input) es **lo que el recording captura**.

Dónde el framework "decide" sin consultar al recording

Hay dos puntos en el código donde el control inventa un throttle **sin consultar** `targetThrottle` del recording:

Punto 1 — `PARKING_PUSH_THROTTLE = 0.35` (hardcoded en parking mode):

```
else if (kmh < 3.0) {
    brake    = 0;
    throttle = 0.35; // empujar hacia el cartel
}
```

Calibrado empíricamente para el bus (necesita 0.35 para vencer inercia estática). Para el Land Rover, 0.35 en primera al salir de una parada técnica = aceleración brutal + posible wheelspin.

Punto 2 — `throttle = 1.0` default en cruise mode:

```
else if (target.hasInputData && target.targetSpeed > 5) {
    if (kmh > target.targetSpeed * 1.05) {
        throttle = 0.3;
    } else if (kmh > target.targetSpeed * 0.9) {
        throttle = 0.6;
    }
    // throttle queda en 1.0 (default) si vamos por debajo
}
```

Cuando `kmh < target.targetSpeed * 0.9`, throttle queda en 1.0 (su valor inicial), **ignorando `target.targetThrottle` del recording**. Para el bus a `target=40 km/h`, `kmh<36` → `throttle=1.0` = aceleración moderada por el motor diesel pesado. Para el Land Rover en primera, 1.0 = wheelspin.

Fix coherente con el principio recording-driven (roadmap v1.x)

Respetar el `targetThrottle` del recording cuando hay desviación moderada, override solo en desviación grande:

```
else if (target.hasInputData && target.targetSpeed > 5) {
    float speedDelta = kmh - target.targetSpeed;
    if (speedDelta > 10) {
        // muy rapido, frenar
        brake = ...; throttle = 0;
    } else if (speedDelta < -10) {
        // muy lento, throttle fondo (override)
        throttle = 1.0;
    } else {
        // dentro de +/-10 km/h del target, RESPETAR EL RECORDING
        throttle = target.targetThrottle;
        brake = target.targetBrake;
    }
}
```

Esto es **más recording-driven puro** que la versión actual. Salvaguarda los casos extremos pero respeta la decisión cognitiva del operador en condiciones normales.

Para el paper — formulación del principio refinado

Anotación del fix al roadmap v1.x

Ya está en el roadmap "vehicle profile completo" — adicionalmente:

- **PARKING_PUSH_THROTTLE** configurable por JSON (default 0.35 bus, 0.15 Land Rover)
- **Cruise default** = `targetThrottle` del recording cuando dentro de ± 10 km/h del target (override solo en desviación grande)
- **Per-gear throttle limit** opcional: si `targetGear` in [primera, segunda], `throttle_max = 0.5` (anti-wheelspin). Pero esto invade el recording-driven — preferir el approach de "respetar targetThrottle" que es más limpio.

Conclusión refinada

Extensión empírica — el gear es otro override silencioso

Tras la corrida de Boris Rover, Sonom4n articuló una segunda instancia del mismo anti-patrón:

Técnica humana avanzada: gear alto + throttle alto = manejo suave

Es la **lógica inversa** de un cambio automático estándar. Una AT por velocidad razona "velocidad baja → gear baja → torque máximo para acelerar". Un operador humano sobre vehículo torquey razona "**si no quiero torque excesivo, subo gear y compenso con throttle**":

- Pendiente con vehículo torquey: gear alta + throttle full = aceleración controlada con potencia (en lugar de gear baja + modulación, que da derrape)
- Engine braking en curva: gear alta sostenida + soltar throttle = freno suave del motor
- Recta de cruce: gear alta + throttle moderado = económico y estable

Esta es **decisión cognitiva contextual** del operador que el cambio automático NO captura. Y es exactamente lo que el operador del Land Rover hizo durante toda la grabación.

El bug del framework

En `BZBusService.c` , el comentario explícito:

```
// NO forzar gear desde el waypoint. La logica AT (RPM-based en
// modded CarScript) maneja los shifts.
```

Esa decisión fue **calibrada para el bus** específicamente. El bus es pesado y dócil, la AT por velocidad funciona OK. Pero el recording sí tiene `target.targetGear` capturado — **lo estamos descartando**.

Resultado en el Land Rover:

```
Operador grabó: en pendiente, gear=6, throttle=0.8 → manejo suave
Framework reproduce: AT shiftea down a gear=2 (por velocidad baja), throttle=0.8 → catapu
```

Mismo throttle, gear DISTINTO → comportamiento físico **opuesto**. El "clavar primera full" que el operador observó es exactamente esto.

Tabla completa de puntos ciegos del framework (extiende 5.27)

Subsistema	Recording tiene	Framework usa	OK en bus?	OK en Land Rover?
Steering target	(posición + heading derivado)	Stanley + corredor	sí	sí (con K calibrado)
Throttle	<code>targetThrottle</code>	hardcoded 1.0 cruise / 0.35 parking	sí	NO (aceleración brutal)
Gear	<code>targetGear</code>	AT por velocidad (RPM-based)	sí	NO (técnica humana ignorada)

Subsistema	Recording tiene	Framework usa	OK en bus?	OK en Land Rover?
Brake	targetBrake	modelo predictivo $u^2/(2s)$ + factor altura	sí	parcial (sobre-correge en pendientes)
Stop position	isStop	sí (m_AtStop)	sí	sí

Dos de cinco subsistemas hacen override silencioso del recording. Ambos calibrados para el bus específicamente.

Fix recomendado (roadmap v1.x prioridad alta)

Volver a usar `target.targetGear` del recording como **deseo primario**:

```
// En lugar del comentario "NO forzar gear desde el waypoint":
if (target.hasInputData && target.targetGear >= 2) {
    // Recording captura la decisión humana de gear – respetar
    m_DesiredGear = target.targetGear;
} else {
    // Fallback: AT RPM-based (recording viejo o sin gear capturado)
    // ... logica actual de gear_ranges.json ...
}
```

La AT por velocidad queda como **fallback solo cuando el recording no tiene gear** (corridas pre-PathLogger). Mismo patrón que en 5.27 con el throttle.

Para el paper — formulación definitiva del principio refinado

Conclusión refinada con corolarios

Esto cierra el paper articuladamente. La sesión completa demostró que el framework opera mejor cuanto MENOS inventa y MÁS reproduce. Es la definición técnica del principio "subordination architecture".

Nota crítica de generalización

Es la formulación **correcta y más fuerte** del bug.

El override del gear NO es vehicle-specific — es **architecture-specific**. El framework asume implícitamente:

Esta asunción es válida **solo si la AT humana del operador es igual a la AT del código**. Si el operador usó decisiones cognitivas no-monotónicas de gear (subir gear para suavizar, mantener gear alta en pendiente, engine braking en curva), el framework las destruye.

Por qué el bus no lo había expuesto VISIBLEMENTE (corrección)

Originalmente atribuimos la falta de síntoma a "la ruta del bus es plana". **Es incorrecto** — la ruta canónica del bus tiene pendientes (Vysotovo-Cherno costera, Solnechny-Nizhnoe, Berezino-Rify), aunque ninguna tan pronunciada como la del Land Rover (delta 86m en 500m).

La formulación correcta: **el síntoma probablemente ESTABA en el bus, pero la masa del vehículo + pendientes moderadas lo atenuaban hasta hacerlo indistinguible de otras desviaciones.**

Aclaración importante :

La validación del factor altura en Sonomir fue un **test SysID** — modo de experimento controlado (NUMPAD 8/9, step y curve response) que **ignora completamente la ruta y el recording**, aplicando inputs programados en línea recta. En ese modo, el override del gear **no se ejecuta**. Por lo tanto:

- Las pendientes que **sí** ejecutarían el bug del gear son las de la **corrida normal del bus** (Vysotovo-Cherno, Solnechny-Nizhnoe, etc.), no las del test SysID
- Esas pendientes son **moderadas**, no pronunciadas como la del Land Rover
- Combinado con la masa del bus, el bug queda **muy atenuado**

Esto refuerza que el bus en su ruta canónica está en un **dominio favorable doble**: bajo masa-impacto (pesado) + bajo pendiente-impacto (suaves). El Land Rover combinó **bajo masa + alta pendiente** = bug manifiesto.

Factor	Bus en ruta canónica	Land Rover en Stary Sobor→Zenit
Pendientes en la ruta	sí (varias)	sí (delta 86m en 500m)
Override de gear ocurre	sí (latente)	sí (manifiesto)
Velocidad de grabación	40-80 km/h	40-80 km/h
Diferencia gear grabado vs AT inferido	sí, pero atenuada por masa	sí, amplificada por torque
Síntoma visible	DEV >2m en wp 4400-4800 que atribuimos a "curvas costeras"	catapulta + zigzag obvio

Re-evaluación de las zonas problemáticas del bus

Mirando el análisis del CSV canónico bajo esta nueva luz: las zonas con `dev > 2m` y velocidad alta (wp 4400-4800, 1000-1049, 1200-1249, 650-699) **pueden ser exactamente el mismo bug del gear**.

Hipótesis: en esas zonas el operador grabó en gear 4 o 5 (porque elegía suavidad o engine braking). La AT por velocidad inferió gear 2-3. El throttle del recording se aplicó en una marcha distinta a la intencionada → catapulta atenuada por masa → **se ve como "se sale del trazo en curvas a alta velocidad"** cuando en realidad es **"la aceleración no coincide con la grabada porque la marcha es distinta"**.

Predicción falsable revisada

Cuando se implemente "respetar `targetGear`", una corrida IA del bus en la ruta canónica **debería**:

- Bajar dev avg global de 1.00m hacia 0.6-0.8m
- Reducir samples con dev>2m del 9.8% a probablemente <3%
- Las zonas hoy problemáticas (wp 4400-4800 etc.) deberían comportarse como las zonas hoy "buenas"

Si la predicción se cumple, **valida retroactivamente que el bus también tenía el bug, solo que era invisible.**

Articulación final — DOS umbrales del framework, no uno

Eso identifica **un segundo umbral** distinto al de discretización temporal (5.25). El framework v1 tiene **dos umbrales paralelos** donde sus bugs latentes se hacen visibles:

Umbral	Magnitud crítica	Forma de cruzar	Síntoma visible
Discretización temporal (5.25)	velocidad × frecuencia_control	velocidad alta + Tick lento	Desvío lateral grande (Stanley no alcanza a corregir)
Respuesta de masa (5.27 extensión)	masa × intensidad_override	masa baja + override sin compensar	Sacudidas longitudinales (catapulta, wheelspin)

El framework v1 opera dentro de un VOLUMEN del espacio de vehículos

No es un punto, es un volumen 2D (al menos):

	Vehiculo: ALCANZA framework v1
velocidad alta	ZONA NO-FRAMEWORK
+ masa baja	(cruza ambos umbrales)

	Land Rover Defender FRONTERA
velocidad media	
+ masa moderada	ZONA FRAMEWORK v1
	(bus en su sweet spot)
velocidad baja	
+ masa alta	ZONA SOBREDIMENSIONADA
	(framework innecesario, vanilla alcanza)

Predicción del comportamiento con vehículos más livianos

Si el Land Rover (~2000 kg) ya cruza el umbral de masa, vehículos más livianos lo cruzarían **más fuerte:**

Vehículo	Masa típica	Predicción del bug del gear sin fix
Bus (ExpansionBus)	~10000 kg	latente, invisible salvo en zonas costeras
V3S (camión militar)	~7000 kg	latente, visible en pendientes pronunciadas

Vehículo	Masa típica	Predicción del bug del gear sin fix
Land Rover Defender	~2000 kg	manifiesto: catapulta + zigzag (lo que vimos)
Hatchback / Olga	~1100 kg	muy manifiesto (probable inmanejable)
Sedan / civilian	~1300 kg	similar

El Land Rover fue el canario en la mina porque está en el primer peldaño bajando. Vehículos más livianos confirmarían que el bug es catastrófico sin el fix.

Implicancia para el paper

Síntesis filosófica del paper completo

Tres principios rectores que emergen acumulativamente de la sesión:

1. ****El framework reproduce comportamiento humano sobre vehículos físicamente válidos**** (5.24: alcance y límites)
2. ****El umbral del framework no es la física del vehículo, sino la discretización temporal y la respuesta de masa**** (5.25 + 5.27 ext.)
3. ****El framework opera mejor cuanto MENOS inventa y MÁS reproduce.**
Cada override silencioso del recording es una asunción de vehículo que limita la generalización** (5.27 principio rector)

La combinación: el framework es **mínimo y delegativo** dentro de su dominio. Para expandir el dominio, no agregar más lógica — **sacar más lógica** (eliminar overrides, respetar más al recording). La ingeniería del framework progresa por **sustracción**, no por adición.

Esa es la conclusión técnica más fuerte del paper.

Tercer punto ciego identificado: sensibilidad de steering (wheelbase)

Físicamente:

$$\text{yaw_rate} \approx \text{velocidad} \cdot \tan(\text{steering_angle}) / \text{wheelbase}$$

A mismo `SetSteering(0.3)` :

- Bus wheelbase ~5-6m → yaw rate moderado
- Land Rover wheelbase ~2.7m → **yaw rate doble** (sobre-rotación)

El framework aplica `steering = dYaw / (π/2)` SIN escalar por vehículo. Para vehículos cortos, las correcciones del Stanley se amplifican geométricamente.

Fix implementado: `SteeringScale` configurable por JSON

Tras `clamp(steering, -1, 1)`, multiplicar por `SteeringScale` del vehicle profile. Valores sugeridos:

Vehículo	Wheelbase	SteeringScale
Bus	~5-6m	1.0 (default, sin escala)
V3S Truck	~4m	0.7
Land Rover Defender	~2.7m	0.5
Hatchback / civilian	~2.5m	0.45
Sedan	~2.7m	0.5

Tabla completa de puntos ciegos del framework (actualización final)

Tres puntos ciegos identificados en la sesión, dos resueltos parcialmente, uno pendiente:

#	Punto ciego	Recording tiene	Framework actual usa	Status del fix
1	Throttle (cruise + parking)	targetThrottle	hardcoded 1.0 / 0.35	PENDIENTE (respetar targetThrottle cuando dentro de ±10 km/h del target)
2	Gear	targetGear (decisión humana contextual)	AT por velocidad/RPM	PARCIAL (anti-catapulta por aceleración medida — ataja desde resultado, no copia input)
3	Steering scale (wheelbase)	(no captura — es propiedad del vehículo)	sin escalar	RESUELTO (SteeringScale configurable por JSON)

Patrón común a los tres: el framework asume implícitamente el perfil físico del bus (peso 10t, wheelbase 5.5m, transmisión diesel con torque dócil). Cada punto ciego se manifiesta cuando el vehículo real se aleja de ese perfil.

Vehicle profile completo (v1.1)

Después de esta sesión, el JSON expone:

```
{
  "VehicleClass":      "Expansion_Landrover",
  "DriverClass":       "eAI_SurvivorM_Boris",
  "MaxGear":           7,
  "Attachments":       [...],
  "AccelShiftThreshold": 15.0,
  "SteeringScale":      0.5,
  "Waypoints":         [...]
}
```

Cinco campos físicos (más Waypoints y clase del driver). Pendiente de v1.x: agregar STANLEY_K , MAX_BRAKE_DECEL , PARKING_PUSH_THROTTLE configurables (los últimos overrides del bus que aún están hardcoded).

Implicancia final para el paper

El Land Rover como canario en la mina

Formulación final para el paper

Predicción falsable para validar el fix

Cuando se implemente "respetar `targetGear` del recording" (roadmap v1.x prioridad alta), correr una corrida IA del bus con la misma ruta canónica. **Predicción:**

- dev avg residual del bus puede bajar de 1.00m hacia 0.7-0.8m
- Las zonas problemáticas a alta velocidad (wp 4400-4800 según análisis previo) **deberían mejorar** porque el operador ahí grabó gear=4-5, no el gear que la AT actualmente infiere
- Las paradas (donde la AT down-shiftea a primera) deberían mantenerse o mejorar (el operador probablemente también grabó primera ahí)

Si la predicción se cumple → confirma que el override actual está degradando incluso al caso de uso "exitoso" (bus en ruta costera). Si no se cumple → la AT por velocidad era una buena aproximación para el bus específicamente, pero igualmente el fix es necesario para generalización.

Cualquiera de los dos resultados es **información valiosa para el paper**.
